

ASD Group 18 Technical Report

<https://github.com/rigels51/ASD-Project-Group18>

1.0 Project Overview

1.1 Project Overview and Objectives

The project is a University Management System developed as a collection of independent microservices. Each group member was responsible for developing an individual university management feature while following a shared architecture and frontend design.

The main objective of the project was to develop a functional integrated application that supports common university management tasks such as managing student records, staff, timetables, courses and enrolments. Each feature was designed with separate frontend, backend/API and database responsibilities to maintain modularity and make the system easier to integrate.

Another major objective was to incorporate local AI functionality into the individual features. The AI Mode allows users to ask natural-language questions about application data, with current database information provided as context to reduce unsupported responses.

The project also aimed to demonstrate modern software development practices, including Agile development, GitHub-based collaboration, Docker containerisation, REST APIs, automated GitHub Actions workflows and the Plan → Act → Observe → Adapt agentic workflow.

1.2 Team Members

Student 1: Rigel 25876487

Student 2: Jeriko Arceo 24806008

Student 3: Lazizbek Ismoilov 14567426

Student 4: Yi Zhang 25402140

Student 5: Vu Tien Thanh Nguyen 14673154

1.3 Individual Feature Allocation (i)

Student 1: Rigel Rivamonte

This feature manages university student records. Users can view, search, create, update, and delete student information. Each record includes a student ID, name, course, year level, email, phone number, GPA, and academic status.

The feature also includes AI Mode, allowing users to ask questions about current student records. The system retrieves live database data before generating contextual answers.

- Display a list of students
- Search for students by name or ID
- View individual student profiles
- Add a new student
- Edit student information
- Delete student records
- AI chat interface for asking questions about student records
- Retrieve all students
- Retrieve a specific student
- Create a student
- Update a student
- Delete a student
- Search for a student by ID

Student 2: Jeriko Arceo

Feature: Staff Management

This feature is responsible for managing staff records for the University management application. The feature allows the user to view, create, edit, and delete staff member details from the university. It also allows staff to be assigned to classes.

Its main functions include:

- Create staff
- Read staff
- Update staff
- Delete staff
- AI agent to assist with assigning staff
- Get staff data
- Get subject data
- Assign staff to classes

Student 3: Lazizbek Ismoilov

Feature: Timetable and Classes Management

This feature manages when and where each course/subject session runs. Students and lecturers can browse and filter the timetable; admin users can create, update, and delete sessions; the system automatically detects room/time scheduling clashes; and a local AI agent answers natural-language scheduling questions grounded in the live timetable data.

- Its main functions include:
- Create a new class session (course code, day, time, room, semester)
- Read / browse all sessions and filter by course code
- Update an existing session's room or time
- Delete a session
- Automatically detect scheduling clashes (same day, room, overlapping time)
- AI agent to answer natural-language scheduling questions, grounded in real timetable data
- Role-based views for Student, Lecturer, and Admin
- Get sessions by course code (cross-feature lookup point for Course Catalogue)

Student4: Yi Zhang

Feature: Course and Enrollment Management

My responsibility was to design and implement the Course and Enrollment Management feature of the University Management System. The feature allows users to view and manage course information and student enrolment records.

The main functions include:

- View all courses and individual course information
- Create new courses
- Update existing courses
- Delete courses when no students are enrolled
- View student enrolment records
- Create new enrolments
- Update enrolment information and status
- Delete or withdraw enrolment records
- Use a local AI agent to answer general questions
- Use database context to answer questions about current courses and enrolments

The feature was implemented using a frontend service, Flask enrolment service, SQLite database service, Docker containers, Ollama AI integration, and GitHub Actions.

Student 5: Vu Tien Thanh Nguyen

Feature: Assessment & Grades Management

My responsibility was to design and implement the Assessment & Grades Management feature of the University Management System. This feature manages course assessments and student results. It allows users to view, search, create, update, and delete assessment and grade records.

The feature also includes an AI Mode that allows users to ask natural-language questions about assessments, grades, and student performance. The AI retrieves current assessment and grade information before generating a response using the Plan → Act → Observe → Adapt workflow.

The main functions include:

- View and search assessments by course, assessment type, or keyword
- View individual assessment details
- Create new assessments
- Update existing assessments
- Delete assessments
- View student grade records
- Record new student grades
- Update and delete grade records
- Search grades by student ID or course
- Validate student IDs using the Student Records Management service
- Retrieve course information from the Course Management service
- Use AI Mode to answer questions about assessments, grades, and student performance
- Use current database information to provide grounded AI responses

The feature was implemented as three separate microservices: an Nginx frontend service, Flask backend/API service, and Flask/SQLite database service. Docker is used to containerise each service, while Ollama with Qwen is used for the local AI functionality.

2.0 Project Analysis and Planning

2.1 Agile Methodology and sprint structure

The project is using an Agile development approach to support the development of the University Management app. Each student was responsible for an individual feature, while the group worked together to integrate the separate services into one University Management System. Everyone made sure all their features are working with one another while also providing active reviewing and feedback for any changes needed for certain mini features within their own features.

Sprints	What will be done	Expected Outcome
Sprint 1	Team formation, topic approval, repository setup, Docker setup, and feature planning	Shared repository and development environment ready
Sprint 2	Database design, frontend development, backend API development, and CRUD implementation	Individual features able to create, read, update, and delete records
Sprint 3	AI Mode integration, applying same CSS design to all pages, Docker containerisation, testing, and shared Docker Compose integration	Release 0 integrated application completed
Sprint 4	Develop MCP and RAG components, connect services to shared retrieval tools, and improve database-grounded AI responses.	MCP and RAG integration is available for Release 1.
Sprint 5	Implement multi-agent workflow, complete testing, collect evidence, resolve outstanding integration issues, and prepare cloud deployment.	Release 2 application, testing evidence, and deployment are completed.

2.2 Sprint Backlog - Functional and Non-Functional Requirements (i)

Functional Requirements	Non-functional Requirements
• The system shall display all student records. • The system shall allow users to search by student name, ID, or course. • The system shall allow users to filter students by status. • The system shall allow users to retrieve student details by ID. • The system shall allow users to create, update, and delete student records. • The system shall generate student IDs using the STU-XXXX format. • The system shall validate required fields before creating records. • The system shall provide AI Mode for questions about student data. • The AI shall use current student records as context before answering. • The frontend shall communicate with the backend through REST endpoints.	• Performance: The student records list and search functions shall load within the range of 2-3 seconds • Usability: The interface shall be simple, consistent with the shared team CSS theme, and usable without the need to teach new users. Users shall be able to search, filter, add, edit, view, and delete student records from the Student Records page. • Reliability: CRUD operations shall return suitable responses and error handling. In the case when a system shall return 400 when required student fields are missing, 404 when a student record cannot be found, and 503 when the database service is unavailable. • Maintainability: The feature shall follow a modular microservice structure with separate frontend, enrolment/application, and database services. • Portability: The Student Records feature shall be fully containerised using Docker and shall run through the shared docker-compose.yml configuration. • Security: Student data shall be stored using parameterised SQLite queries to reduce SQL injection risk. Required user input shall be checked before records are written to the database. • Scalability: The SQLite students table shall support growth beyond the initial

	twelve student records without requiring database schema changes. • Availability: The frontend, enrolment service, and database service shall be available whenever the Docker Compose stack is running. Docker restart policies shall help services restart automatically after unexpected container failures. • Interoperability: The backend shall expose REST API endpoints using JSON responses so that other group services can retrieve student data by student ID or course without accessing the SQLite database directly. • AI Reliability: The local AI assistant shall retrieve current student records before answering contextual questions. Its prompt shall instruct the Ollama model to use only supplied student data and not invent student details. • Testability: The feature shall support automated unit testing for core student-record functions. GitHub Actions shall validate Python syntax and build Docker images through the RigelRivamonte.yml workflow.
--	---

Jeriko Arceo

Functional Requirements	Non-functional Requirements
• The system shall allow users to create a new staff record (e.g. staff ID, name, email, department, employment type)	• Performance – The staff list page shall load within 2–3 seconds under normal local Docker execution.

• The system shall allow users to view/read a list of all staff members and view individual staff details. • The system shall allow users to update existing staff records • The system shall allow users to delete a staff record from the system. • The system shall validate staff input data (e.g. required fields, valid email format, unique staff ID) before saving to the database. • The system shall store and retrieve staff data from a dedicated Staff database • The frontend shall communicate with the backend/API via REST endpoints (GET, POST, PUT/PATCH, DELETE) to perform CRUD operations. • The system shall integrate with the AI-Mode to allow AI-assisted interaction — e.g. a natural-language query such as "list all staff in the Computer Science department" or "summarise staff workload by department." • The system shall display staff data in the unified frontend UI, following the shared CSS theme and accessible from the unified home page • The system shall provide search/filter functionality for staff • The system shall demonstrate the Plan → Act → Observe → Adapt agentic workflow when processing an AI-assisted staff-related request. • The system shall allow users to update/reassign a staff member's course allocation (e.g. change or add a course assignment).	• Usability – The UI shall be simple, consistent with the shared team CSS theme, and usable without training. • Reliability – CRUD operations shall complete successfully with appropriate error handling and user feedback (e.g. failed save, duplicate ID). • Maintainability – Code shall follow modular structure (separate frontend, backend, database layers) to support incremental extension in Release 1/2. • Portability – The microservice shall be fully containerised with Docker and run successfully via docker-compose up alongside other services. • Security – Input shall be sanitised to prevent SQL injection/XSS; sensitive staff data (e.g. contact info) shall not be exposed in logs. • Scalability – The database design shall support growth beyond the initial 10 records without requiring schema changes. • Availability – The microservice shall be available whenever the Docker Compose stack is running, with no manual intervention needed. • Interoperability – The backend/API shall expose REST endpoints in a consistent format so other student microservices (or future MCP/RAG integration) can consume staff data if needed. • Testability – The microservice shall support GitHub Actions CI validation (build/test) via student-2.yml.
--	---

Lazizbek Ismoilov

Functional Requirements	Non-functional Requirements
-------------------------	-----------------------------

• The system shall allow Admin users to create a new timetable session (course code, session type, day, start time, end time, room, semester) • The system shall allow users to view/read all sessions and filter by course code • The system shall allow Admin users to update an existing session's room, start time, or end time • The system shall allow Admin users to delete a session from the timetable • The system shall automatically detect scheduling clashes (same day, same room, overlapping time range) • The system shall restrict Create/Update/Delete/Clash-check actions to the Admin role; Student and Lecturer roles are read-only (browse, filter, ask AI) • The frontend shall communicate with the backend/API via REST endpoints (GET, POST, PUT, DELETE) to perform CRUD operations • The system shall integrate with AI-Mode to answer natural-language scheduling questions, grounded in the live timetable data retrieved from the database • The system shall display timetable data in the unified frontend UI, following the shared CSS theme and accessible from the unified home page • The system shall demonstrate the Plan → Act → Observe → Adapt agentic	• Performance — GET /timetable shall respond within 500ms under normal local Docker execution (validated: average ~2.7ms across 20 sequential requests) • Usability — The UI shall follow the shared team CSS theme (ink/paper/brass tokens) and be usable without training • Reliability — CRUD operations shall complete successfully with appropriate error handling (400 for missing fields, 404 for not-found records) • Maintainability — Code shall follow modular structure (separate frontend, backend, database layers) to support incremental extension in Release 1/2 • Portability — The microservice shall be fully containerised with Docker and run successfully via docker-compose up alongside other services • Security — Input shall be validated before database writes; error messages shall not expose internal implementation details • Scalability — The database design shall support growth beyond the initial 10 records without requiring schema changes • Availability — The microservice shall be available whenever the Docker Compose stack is running, with no manual intervention needed • Interoperability — The backend/API shall expose REST endpoints (course_code, staff_id fields) in a
--	--

workflow when processing an AI-assisted scheduling request	consistent format so other student microservices (Course Catalogue, Staff Management) can integrate • Testability — The microservice shall support GitHub Actions CI validation via student-3.yml, and passed a 16-check local validation suite covering all endpoints
--	---

Yi Zhang

Functional Requirements	Non-functional Requirements
● The system should display all available courses. ● The system should allow users to retrieve information about a specific course. ● The system should allow new courses to be created. ● The system should allow existing course information to be updated. ● The system should allow courses to be deleted when appropriate. ● The system should display student enrolment records. ● The system should allow students to be enrolled in courses. ● The system should allow enrolment information and status to be updated. ● The system should allow enrolment records to be removed or withdrawn. ● The system should provide an AI Mode for natural-language questions. ● The AI should be able to use current course and enrolment information from the database.	● The system was designed using a microservice structure, where the frontend, enrolment service and database service have separate responsibilities. ● Docker was used to containerise the services so that the feature could be integrated with the other group features. ● User input is validated before database operations are completed. For example, required fields and numeric values are checked before records are created or updated. ● The system also returns suitable HTTP error responses when an operation fails. ● The database maintains the relationship between courses and enrolments through the course ID. ● The AI function runs locally using Ollama instead of depending on an external online AI service. ● GitHub Actions was also used to automatically validate Python code, Docker builds and Docker Compose configuration.

Functional Requirements	Non-functional Requirements
• The system shall display all assessment records. • The system shall allow users to search and filter assessments by course, assessment name, and assessment type. • The system shall allow users to retrieve individual assessment details. • The system shall allow assessments to be created, updated, and deleted. • The system shall display grade records. • The system shall allow grades to be retrieved by student ID and course. • The system shall allow grade records to be created, updated, and deleted. • The system shall validate student IDs through the Student Records service before creating or updating grade records. • The system shall retrieve course information from the Course Management service. • The system shall provide an AI Mode for natural-language questions about assessments and grades. • The AI shall use current assessment and grade information before answering questions.	• Usability: The Assessment & Grades interface shall follow the shared team design and provide clear interfaces for viewing and managing assessment and grade records. • Reliability: CRUD operations shall return suitable HTTP responses and error handling when records cannot be found, input is invalid, or another required service is unavailable. • Maintainability: The feature shall use a modular microservice structure with separate frontend, backend/API, and database services. • Portability: The Assessment & Grades feature shall be containerised using Docker and operate as part of the shared Docker Compose application. • Security: User input shall be validated before assessment or grade information is written to the database. • Data Integrity: Grade records shall reference valid assessment records, and student IDs shall be validated through the Student Records service before grade creation or update. • Availability: Standard assessment and grade CRUD functionality shall remain separate from AI Mode so that normal operations can continue if Ollama is unavailable. • Interoperability: The feature shall communicate with other microservices using REST APIs and the shared Docker network. • AI Reliability: AI responses shall be grounded in retrieved assessment and grade information, and the model shall be instructed not to invent unsupported information. • Testability: Backend endpoints shall be validated using automated tests and CI/CD checks. • Persistence: Assessment and grade data shall be stored in SQLite using a named Docker volume so that data can persist across container restarts.

2.3 Feature Plan (i)

Rigel Rivamonte

Feature Name: Student Records Management

The Student Records Management feature was planned as a separate microservice group within the University Management System. The main aim was to provide a complete student-record workflow while keeping the frontend, application logic, and database access separated.

Stage 1 - Data Design

The first stage was to identify the information required for each student record. The Student entity contains a student ID, full name, course, year level, email address, phone number, GPA, and academic status. The student ID is stored as a text primary key using the format STU-XXXX. This gives every student a unique identifier and makes it possible to retrieve individual records consistently through the API.

Stage 2 - Database Service

A Flask database service using SQLite was developed to manage student records. The database contains a student's table and is seeded with twelve sample records for testing and demonstration.

The database service provides endpoints to:

- Retrieve all students.
- Search for students.
- Retrieve one student with an ID.
- Retrieve students by course.
- Create students.
- Update students.
- Delete students.

Stage 3 - Enrolment/Application Service

A separate Flask service was created as the application layer. The frontend sends requests to this service, and the application service communicates with the database service through HTTP requests.

This separation prevents the frontend from accessing the database directly. It also provides a suitable place for error handling, AI integration, CORS support, and future business rules.

Stage 4 - Frontend Development

The frontend was developed as an HTML, CSS, and JavaScript interface served through nginx. It includes a student records table, search feature, status filters, student statistics, course summary, detail drawer, add form, edit form, delete confirmation, and AI chat interface.

The frontend is linked from the group's shared home page and follows the team's university management visual style.

Stage 5 - AI Mode

A local Ollama model using qwen2.5:0.5b was integrated into the application service. Common questions are answered using deterministic program logic. Broader questions are sent to Ollama with the current student records included as prompt context.

The contextual prompt explicitly instructs the model to use only the supplied records, copy names and IDs exactly, avoid inventing data, and state when information is unavailable.

Stage 6 - Docker Integration

The Student Records feature is containerised as the Frontend service, Enrolment service and the Database service. The services are connected through the shared Docker bridge network and integrated into the root docker-compose.yml file.

Stage 7 - CI/CD

A GitHub Actions workflow named RigelRivamonte.yml was created for Student 1. The workflow installs dependencies, validates Python syntax, and builds Docker images for the frontend, enrolment service, and database service.

[Jeriko Arceo](#)

Feature Name: Staff Management

Description: Allows university administrators to manage staff records and assign staff to the courses they teach. Supports full CRUD on staff data and on staff-course assignments, with AI-assisted queries

Frontend: HTMX + HTML/CSS/JS, following the shared team theme. Pages for staff list (searchable/filterable), staff detail (with assigned courses), add/edit staff form, and an assign-course interface. Linked from the unified home page.

Backend/API: Python + Flask REST API. Handles CRUD for staff and for staff-course assignments (assign, view, remove). Validates input, prevents duplicate/invalid assignments. Connects to Ollama/LLM for AI-assisted staff/course queries as part of the Plan → Act → Observe → Adapt workflow.

Database: SQLite. Key tables: staff, courses (if not owned elsewhere), and staff_course_assignment (junction table linking staff and courses, with foreign keys). Seeded with at least 10 staff records.

CRUD Summary:

- Create/Read/Update/Delete staff records
- Create/Read/Delete staff-course assignments (update = reassign)

AI Integration: Natural-language queries (e.g. "list staff in Computer Science") go through the backend to the LLM, which interprets the request and returns matching staff/assignment data.

Dependencies: Needs to confirm with the Course feature owner how course data is shared (shared table vs. API call). Relies on the shared AI-Mode service and shared Docker Compose setup.

[Lazizbek Ismoilov](#)

Feature Name: Timetable & Class Scheduling

Description: Manages when and where each course/subject session runs. Supports full CRUD on session records (Admin role), automatic clash detection, role-based views (Student/Lecturer/Admin), and AI-assisted natural-language scheduling queries.

Frontend: HTML/CSS/JS following the shared team theme (ink/paper/brass tokens, sidebar panel navigation, drawer pattern for the Create form, pill components for session type and clash status). Sections for Browse, Manage (Admin only), Clashes (Admin only), and Ask AI. Linked from the unified home page.

Backend/API: Python + Flask REST API. Endpoints for CRUD on timetable sessions, a clash-detection endpoint comparing day/room/time overlap across all sessions, and an AI-mode endpoint that retrieves live timetable data and injects it into the LLM prompt as grounding context before answering.

Database: SQLite. Table: timetable (session_id, course_code, staff_id, session_type, day, start_time, end_time, room, semester). Seeded with 10 sessions across 5 courses. staff_id is nullable, ready to reference Staff Management once integrated.

CRUD Summary:

- Create/Read/Update/Delete timetable sessions (Admin)
- Read-only browse and filter (Student, Lecturer)

AI Integration: Natural-language scheduling questions are answered using the current timetable data as grounding context, following the Plan → Act → Observe → Adapt workflow, via Ollama (qwen2.5:0.5b).

Dependencies: course_code aligns with the Course Catalogue feature once available; staff_id aligns with Staff Management's staff records (format confirmed with Jeriko Arceo). Relies on the shared AI-Mode service and shared Docker Compose setup.

Yi Zhang

The Course and Enrolment Management feature was developed in several stages.

Stage 1 – Data Design

The first stage was to identify the data required for courses and enrolments.

The Course entity contains information such as the course ID, course code, course name, credits and capacity.

The Enrolment entity contains information such as the enrolment ID, student ID, course ID and enrolment status.

The course ID is used to connect each enrolment record to a course.

Stage 2 – Database Service

A Flask database service using SQLite was developed to store and manage the data.

CRUD operations were created for both courses and enrolments.

The database was also populated with initial sample course and enrolment records for testing and demonstration.

Stage 3 – Enrolment Service

A separate Flask enrolment service was created to act as the application layer.

The frontend sends requests to this service, and the enrolment service communicates with the database service through HTTP requests.

This keeps the frontend separate from direct database access.

Stage 4 – Frontend Development

The frontend was developed to provide interfaces for Course Management, Student Enrolment and AI Mode.

Users can complete normal course and enrolment operations through the interface instead of using the API directly.

Stage 5 – AI Mode

A local Ollama model was integrated into the feature.

The normal AI function allows users to ask general questions about Course and Enrollment Management.

A second AI function uses current database information before answering the user.

For example, the user can ask which courses have available seats. The system first retrieves the current course and enrolment information and then provides this information to the AI model.

Stage 6 – Docker Integration

The frontend, enrolment service and database service were containerised.

The Student 4 services were then connected to the final group Docker Compose application.

The Course and Enrollment Management feature can be accessed through the integrated group system.

Stage 7 – CI/CD

A GitHub Actions workflow was created for the Student 4 feature.

The workflow checks Python syntax, builds the Docker images and validates the Docker Compose configuration.

Vu Tien Thanh Nguyen

The Assessment & Grades Management feature was developed in several stages. The main aim was to provide a complete system for managing course assessments and student grades while keeping the frontend, backend/API, and database services separated.

Stage 1 – Data Design

The first stage was to identify the data required for assessments and grades.

The Assessment entity contains assessment ID, course ID, assessment name, assessment type, description, due date, maximum mark, and weight.

The Grade entity contains grade ID, assessment ID, student ID, mark, grade, feedback, and date recorded.

The assessment ID connects each grade to an assessment. The student ID follows the same STU-XXXX format used by Student 1, while the course ID is aligned with the course codes used by Student 4.

Stage 2 – Database Service

A Flask database service using SQLite was developed to store and manage assessment and grade data.

CRUD operations were implemented for both assessments and grades. The database was also populated with sample assessment and grade records for testing and demonstration.

The database service provides endpoints to:

- Retrieve, create, update, and delete assessments
- Search and filter assessments
- Retrieve, create, update, and delete grades
- Retrieve grades by student ID

- Retrieve grades by course ID

Stage 3 – Backend/API Service

A separate Flask backend service was created as the application layer.

The frontend communicates with this service, and the backend communicates with the Assessment & Grades database service through REST API requests.

The backend also communicates with Student 1's Student Records service to validate student IDs and retrieve student names. It communicates with Student 4's Course Management service to retrieve course information.

Stage 4 – Frontend Development

The frontend was developed using HTML, CSS, JavaScript, and Nginx.

The interface provides separate Normal UI and AI Mode tabs. The Normal UI allows users to view, search, create, edit, and delete assessment and grade records.

The frontend follows the shared group design and is accessible from the integrated University Management System home page.

Stage 5 – AI Mode

A local Ollama model using qwen2.5:0.5b was integrated into the backend service.

The AI Mode allows users to ask natural-language questions about assessments, grades, and student performance. The backend retrieves relevant assessment and grade information and provides it as context to the AI model.

The AI follows the Plan → Act → Observe → Adapt workflow and is instructed to use the supplied data rather than invent unsupported assessment or grade information.

Stage 6 – Docker Integration

The Assessment & Grades feature was containerised into three services: frontend, backend/API, and database.

The services were connected through the shared Docker network and integrated into the group's Docker Compose application.

The feature uses the `assessment_grades_data` Docker volume to provide persistent SQLite database storage.

Stage 7 – CI/CD

A GitHub Actions workflow named `VuTienThanhNguyen.yml` was created for the Student 5 feature.

2.4 Risk Management Plan (i)

Rigel Rivamonte

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	The database service may be unavailable when the application service attempts to retrieve student records.	M	H	Use Docker Compose dependencies, service names, and request timeouts.	Return a clear 503 service-unavailable response to the frontend.
2	Ollama may not be running or the required model may not be installed.	M	M	Configure the Ollama URL and model through environment variables and test AI Mode locally.	Keep deterministic answers and standard CRUD/search functions available.
3	Cross-origin frontend requests may fail because the frontend and Flask service use different ports.	M	H	Enable Flask-CORS in the enrolment/application service.	Review CORS settings and return readable frontend error messages.
4	Invalid or incomplete student data may be sent	M	M	Validate required fields before creating a record.	Return an HTTP 400 response explaining which

	during record creation.				fields are missing.
5	Duplicate or inconsistent student IDs may be created.	L	M	Use the SQLite primary key and generate new IDs from the highest existing student number.	Correct the record manually and reseed the development database if required.
6	SQLite data may be lost when a container is recreated.	L	H	Use the named Docker volume student1_database_data.	Rebuild and reseed the database using init_db.py.
7	Docker build paths may fail during group integration.	M	M	Test each Dockerfile using the same build context configured in Docker Compose.	Build services individually to identify and correct the failing Docker path.
8	Merge conflicts may occur in the shared docker-compose.yml.	M	M	Keep feature code inside student-1 and make small focused commits.	Resolve the shared compose file manually while preserving all team service definitions.
9	AI answers may contain unsupported details if the prompt is too broad.	M	M	Pass live records into the prompt and explicitly instruct the model not to invent information.	Return a deterministic answer where supported or state that the data is unavailable.

Jeriko Arceo

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Dependency on Course data owned by another student — schema or API mismatch could break	H	H	Agree early with the Course feature owner on a shared schema/API	If integration isn't ready in time, use a local mock/stub courses table with sample data so

	staff-course assignments			contract (e.g. course_id format, whether to call their API or share a table)	Staff Management can still be demoed independently
2	AI-Mode / Ollama integration fails or produces unreliable responses for staff/course queries	M	M	Test AI-Mode early with simple prompts; keep query scope narrow (e.g. one or two supported query types)	Fall back to standard UI search/filter if AI response fails, so core CRUD still works
3	Docker networking/integration issues when combining with teammates' microservices in shared docker-compose.yml	M	H	Test the microservice in isolation first, then integrate early and often rather than waiting until the deadline	Keep a working standalone Docker Compose version as backup for individual demonstration
4	Data integrity issues in the staff-course assignment table (e.g. duplicate or orphaned assignments)	M	M	Add validation checks and foreign key constraints in the database schema	Add a manual data-cleanup script to reset/reseed data before demos
5	Time constraints — assignment feature (many-to-many relationship) takes longer to build than standard CRUD	M	M	Prioritise core staff CRUD first (must-have), treat assignment feature as a stretch goal within Release 0 sprint	If time runs out, submit staff CRUD complete and document assignment feature as a known limitation
6	GitHub Actions workflow (student-x.yml) fails due to environment/config errors	L	M	Test workflow file locally and in a feature branch before merging to main	Debug using workflow logs; keep last known working workflow version to revert to
7	Merge conflicts with teammates when	M	L	Commit and push regularly in	Resolve conflicts collaboratively; use

	integrating into shared repository			small increments; communicate with team before major structural changes	feature branches and pull requests rather than direct commits to main
--	------------------------------------	--	--	---	---

Yi Zhang

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Student 4 cannot connect to the Student 1 API when validating a student ID	M	H	Use the shared Docker network and the Student 1 service name in STUDENT_SERVICE_URL	Return a clear error message and stop the enrolment from being created
2	Student IDs may use an incorrect format or may not exist in Student 1 records	M	H	Use the same STU-XXXX format as Student 1 and validate the ID through the Student 1 API	Reject the enrolment and ask the user to enter a valid student ID
3	Invalid course or enrolment input may cause database errors	M	M	Validate required fields and numeric values before creating or updating records	Return a 400 error with a clear message so the user can correct the input
4	Ollama may be unavailable or the local model may not respond	M	M	Keep AI Mode separate from normal CRUD functions and configure Ollama through environment variables	Continue using Course and Enrollment CRUD functions without AI Mode
5	Docker service or container conflicts may prevent the feature from starting correctly	M	M	Use consistent service names, ports, and the shared Docker Compose network	Run docker compose down, remove conflicting containers if needed, and rebuild the services
6	Course and enrolment data	L	H	Use course_id as a foreign key and validate that a course exists	Reject invalid enrolments and rebuild or reseed

	may become inconsistent			before creating an enrolment	the database if required
7	Merge conflicts may occur when Student 4 changes are integrated into the shared repository	M	M	Use the Student 4 feature branch, make small commits, and review changes before merging	Resolve conflicts manually while keeping the latest Student 4 and shared integration changes

Lazizbek Ismoilov

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Flask backend defaults to binding 127.0.0.1, which is unreachable once containerised	M	H	Explicitly bind host="0.0.0.0" in app.run() before containerising	Caught via standalone Docker testing before integration; documented in evidence log
2	Frontend and backend run as separate origins (different ports) once split into microservices, causing CORS/cross-origin request failures	M	H	Enable flask-cors on the backend early, before building frontend features that call it	Add CORS headers manually via after_request if flask-cors is unavailable
3	htmx 2.x blocks cross-origin AJAX requests by default (selfRequestsOnly), causing silent frontend failures	L	M	Test each fetch/AJAX call against the real cross-origin setup, not just same-origin during development	Replace htmx.ajax() calls with plain fetch() for cross-origin requests
4	Docker container name conflicts when switching between standalone and shared docker-compose runs	M	L	Use docker compose down (not just Ctrl+C) when switching contexts	Force-remove conflicting containers by name (docker rm

				between standalone and shared stacks	-f) before retrying
5	Merge conflicts in shared docker-compose.yml when multiple students' services are integrated close together	H	M	Commit and push service additions in small increments; communicate before major structural changes	Resolve conflicts by merging both services' blocks manually rather than picking one side
6	Dependency on Staff Management's staff_id format for the staff_id foreign key field	M	L	Keep staff_id as a nullable generic TEXT field until format is confirmed with the feature owner	Ship Release 0 with staff_id unpopulated; document as a known integration point for Release 1
7	Time constraints — building a fully role-based UI (Student/Lecturer/Admin) alongside CRUD and AI-mode takes longer than a flat single-role UI	M	M	Prioritise core CRUD and clash detection first (must-have), treat full role-based view polish as a stretch goal within the Release 0 sprint	If time runs out, ship with a simpler role toggle and document remaining polish as a known limitation

Vu Tien Thanh Nguyen

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Student Records service is unavailable when validating a grade's student_id	M	H	Use REST API communication and appropriate exception handling when validating students	Return a clear 503 response and prevent an invalid/unverified grade from being recorded

2	Course Management service is unavailable	M	M	Handle failed course lookups gracefully	Display the course ID without the course name until the service becomes available
3	Ollama or Qwen model is unavailable	M	M	Keep normal CRUD functionality independent from AI Mode	Return a clear AI service error while keeping Assessment & Grades CRUD available
4	Invalid assessment or grade data is submitted	M	M	Validate required fields, numeric fields, student ID format and referenced assessment IDs	Return appropriate 400/404 responses without saving invalid records
5	Docker services fail to communicate	M	H	Use the shared enrolment-network, service names and environment variables for service discovery	Check container status, ports, network configuration and service URLs
6	Merge conflicts occur during integration	M	M	Keep Student 5 code within the student-5/ folder and integrate shared files carefully	Resolve conflicts manually while retaining required service configurations
7	AI generates unsupported assessment or grade information	M	H	Supply retrieved database evidence to the model and instruct it not to invent data	Return that information is unavailable when the retrieved evidence does not contain the answer

8	Changes introduce errors into the Student 5 services	M	M	Use automated tests and GitHub Actions validation	Fix failed tests/builds before merging changes
---	--	---	---	---	--

2.5 Data Design

The University Management System uses a distributed data design. Each feature manages its own data through a separate database service, while the different microservices communicate through REST APIs.

The main data areas include student records, staff records, course and enrolment data, timetable information, and assessment and grade information.

2.5.1 Conceptual Data Model

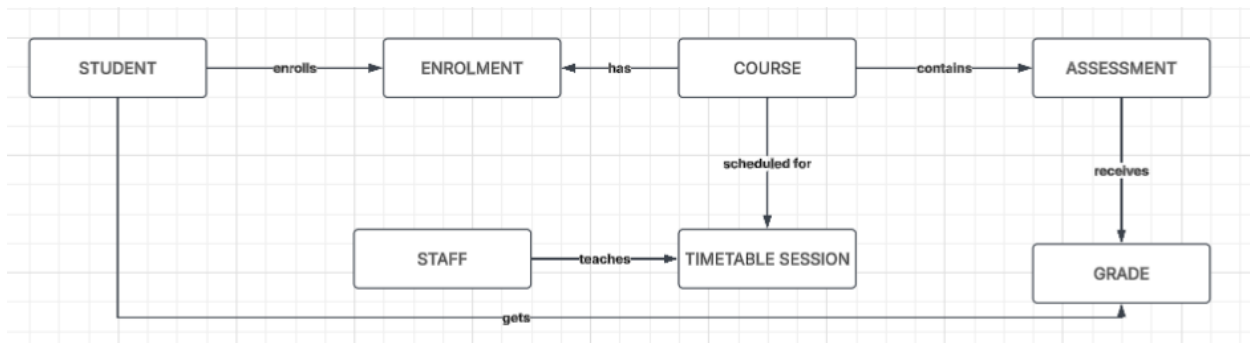
The conceptual data model identifies the main entities in the system and their relationships.

The main entities include:

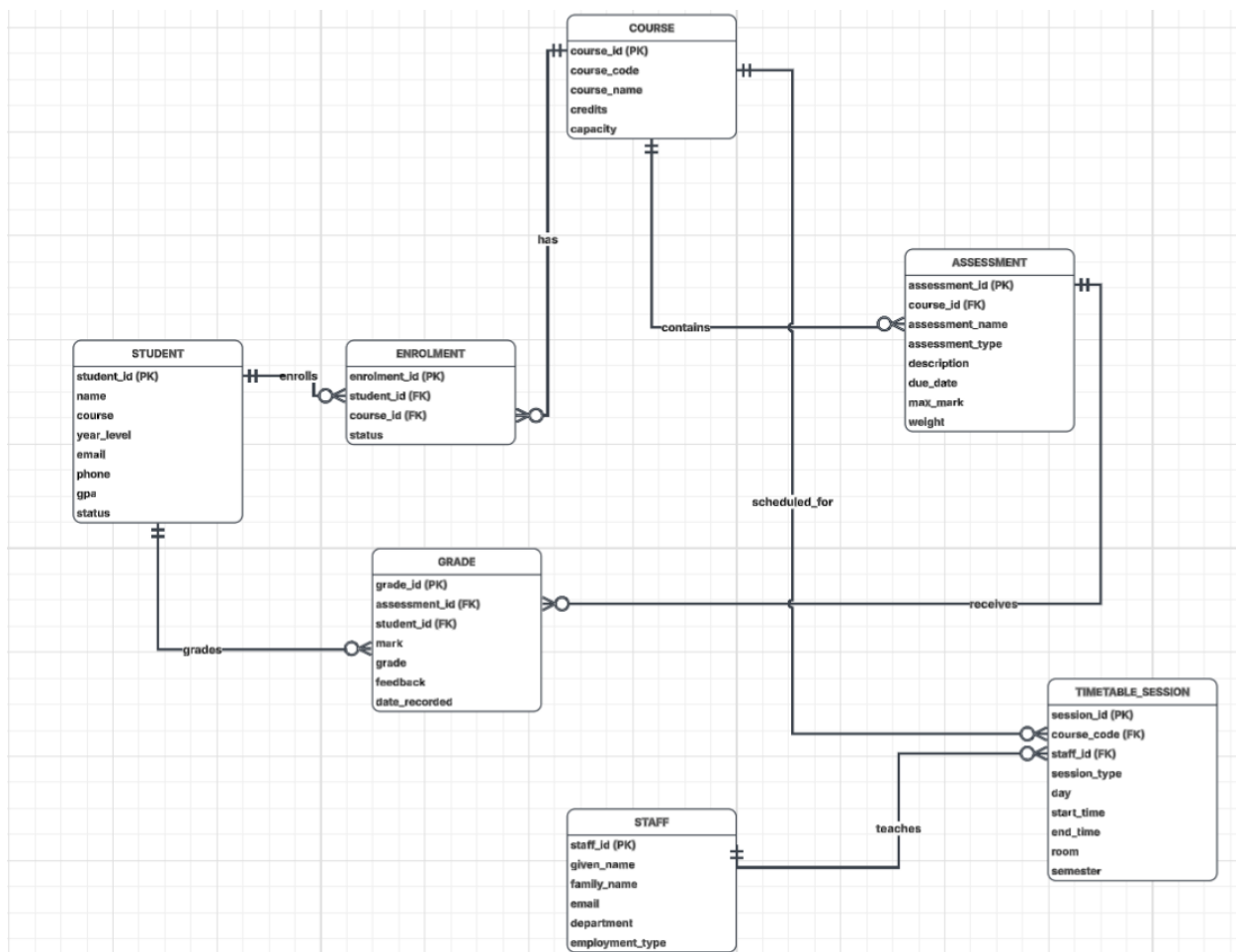
- Student
- Staff
- Course
- Enrolment
- Timetable Session
- Assessment
- Grade

Students can enrol in courses, courses can have timetable sessions and assessments, and staff can be linked to teaching activities. Grades are used to record student results for assessments.

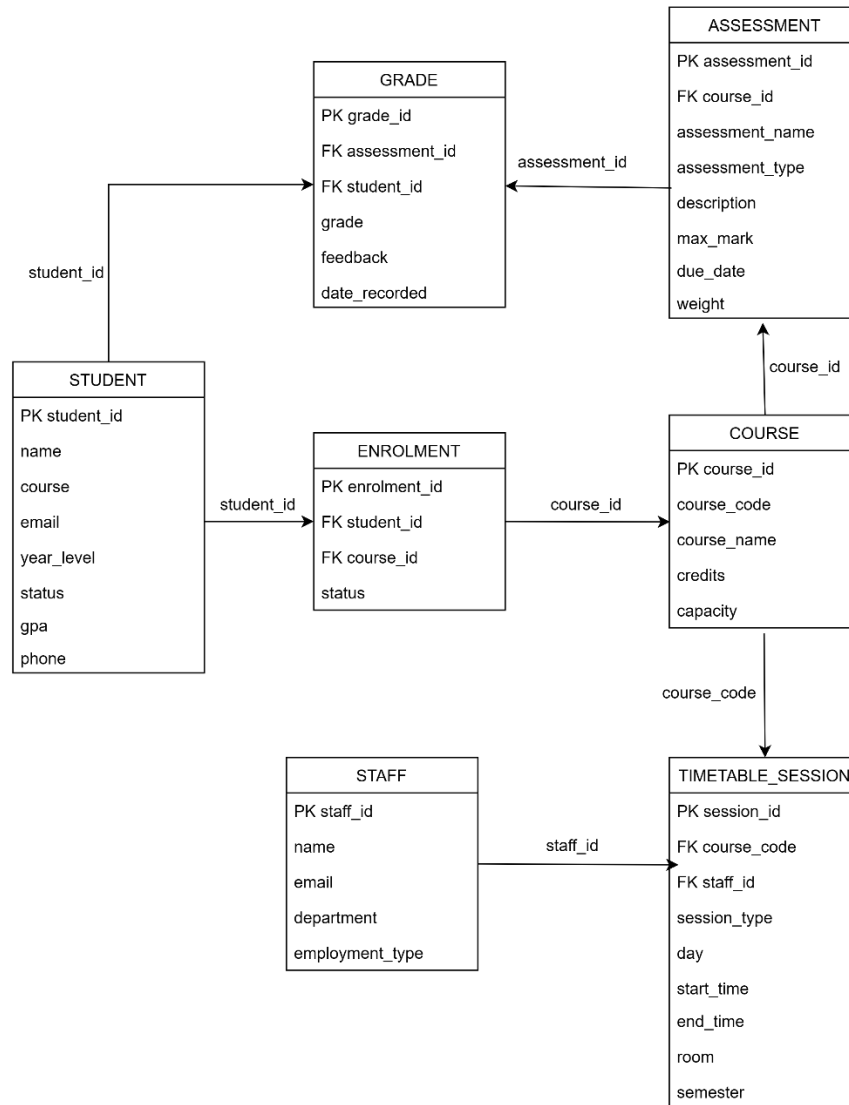
These relationships provide the basic data structure for the integrated University Management System.



2.5.2 ERD



2.5.3 Logical Data Model



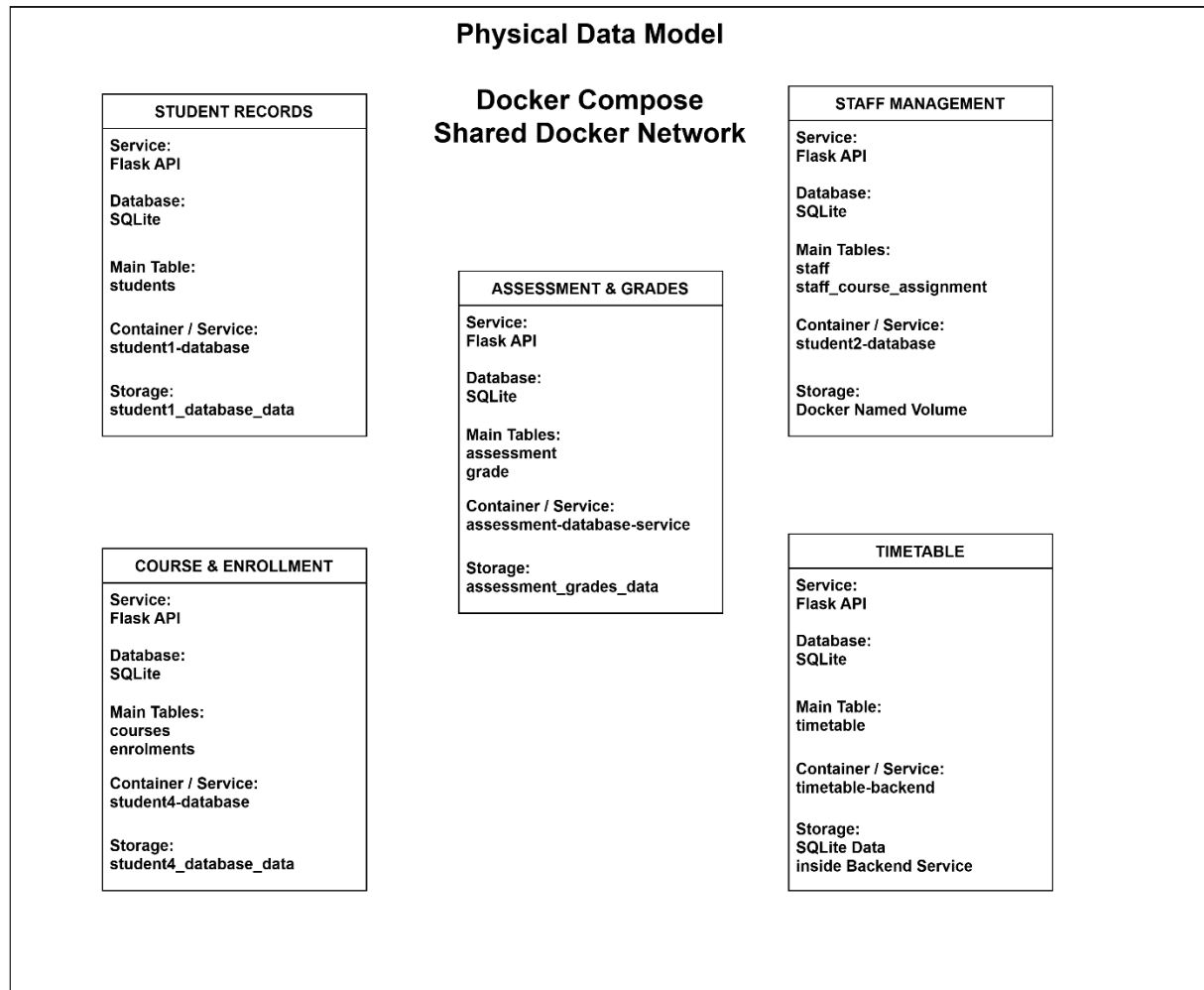
2.5.4 Physical Data Model

The physical data model shows how the system data is implemented in the final application. Each feature uses a separate service and database structure, with SQLite used for data storage.

The Student Records, Staff Management, Course and Enrollment, Timetable, and Assessment and Grades features are deployed as separate Docker services. The services run together through the shared docker-compose.yml and shared Docker network.

Each database service manages its own data, and persistent storage is provided through Docker volumes where required. Cross-feature communication is handled through REST APIs instead of directly accessing another service's database.

This design keeps the system modular and allows each feature to be developed, tested, and integrated independently.

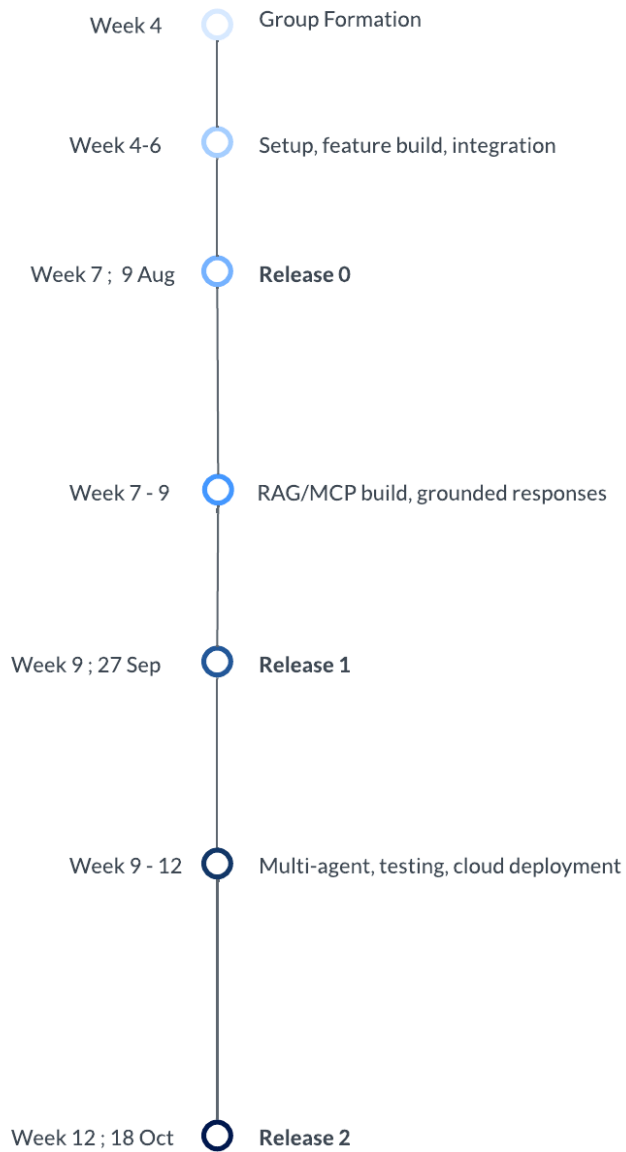


2.6 Overall Project Plan (Timeline/Gantt/Milestones)

Timeline

Project Timeline

University Management Application



Gantt Chart

	W4	W5	W6	W7	W8	W9	W10	W11	W12
Team Formation									
Setup									
Feature Build									
AI Integration									
MCP + RAG build									
Multi-agent, cloud									



Release 0



Release 1



Release 2

Milestones

	Milestone	Week	Date	Description
1	Team formation & topic approval	4		Group finalised, Canvas group formed, topic and individual features approved by tutor
2	Repository & environment setup complete	5		Shared GitHub repo created with standard structure; Docker, Ollama, and dev environments configured for all members
3	Individual CRUD functionality complete	6		Each student's frontend/backend/database microservice supports Create, Read, Update, Delete independently
4	AI-Mode integration complete	6-7		Each microservice successfully calls Ollama + approved LLM
5	Release 0 submission	7	9 Aug	Integrated app, Docker Compose, GitHub Actions workflows, technical report, and showcase video submitted

6	MCP & RAG servers integrated	8		Shared MCP and RAG servers built and connected to individual microservices
7	Grounded AI responses functional	8-9		AI responses demonstrably use retrieved context (RAG) across the integrated app
8	Release 1 submission	9	27 Sep	Updated integrated app, updated report, and showcase video submitted
9	Multi-Agent System integrated	10-11		Planner, Worker, and Reviewer agents functioning within the shared workflow
10	Pre/post-testing complete	11		Unit, integration, and system testing evidence collected for all microservices
11	Cloud deployment successful	11-12		Integrated app deployed to Azure or AWS with AI-Mode enabled, MCP/RAG/Multi-Agent disabled
12	Release 2 submission	12	18 Oct	Final integrated app, complete report, and showcase video submitted

3.0 Repository Structure

3.1 Shared GitHub Repository Layout

The repository (ASD-Project-Group18) is organised, so each student's feature is fully self-contained, with shared, team-wide resources kept separate from individual work.

Top-level layout:

- .github/workflows/ — one CI/CD workflow file per student (e.g. student-3.yml), each scoped to only trigger on changes to that student's own folder
- shared/ — the unified home page (homepage.html), shared CSS theme, and its Dockerfile
- student-1/ through student-5/ — one folder per student, each containing that student's frontend, backend/API, and database code
- docker-compose.yml — the single shared file orchestrating all services across all five features plus the shared home page
- README.md — project overview and per-feature setup instructions

Each student-N/ folder follows a three-tier internal structure, though the exact naming varies slightly by student (e.g. frontend/backend/database for Student 3 and Student 5, versus frontend-service/enrolment-service/database-service for Students 1, 2, and 4) — reflecting that folder-level naming was standardised to student-1...student-5 as a group during Release 0, while internal sub-folder naming was left to each student's own convention.

3.2 Branching Strategy and Folder Organization

Each student develops on their own feature branch (e.g. student-3/timetable-integration, jeriko, Yi-Zhang, rigel) and merges into main via Pull Request, following the spec's required PR-based workflow rather than direct commits to main.

Folder organisation directly enforces the microservice boundary rule: a student's code only ever lives inside their own student-N/ folder, and cross-feature data is accessed only via that feature's REST API, never by reading another student's database file or importing their code directly.

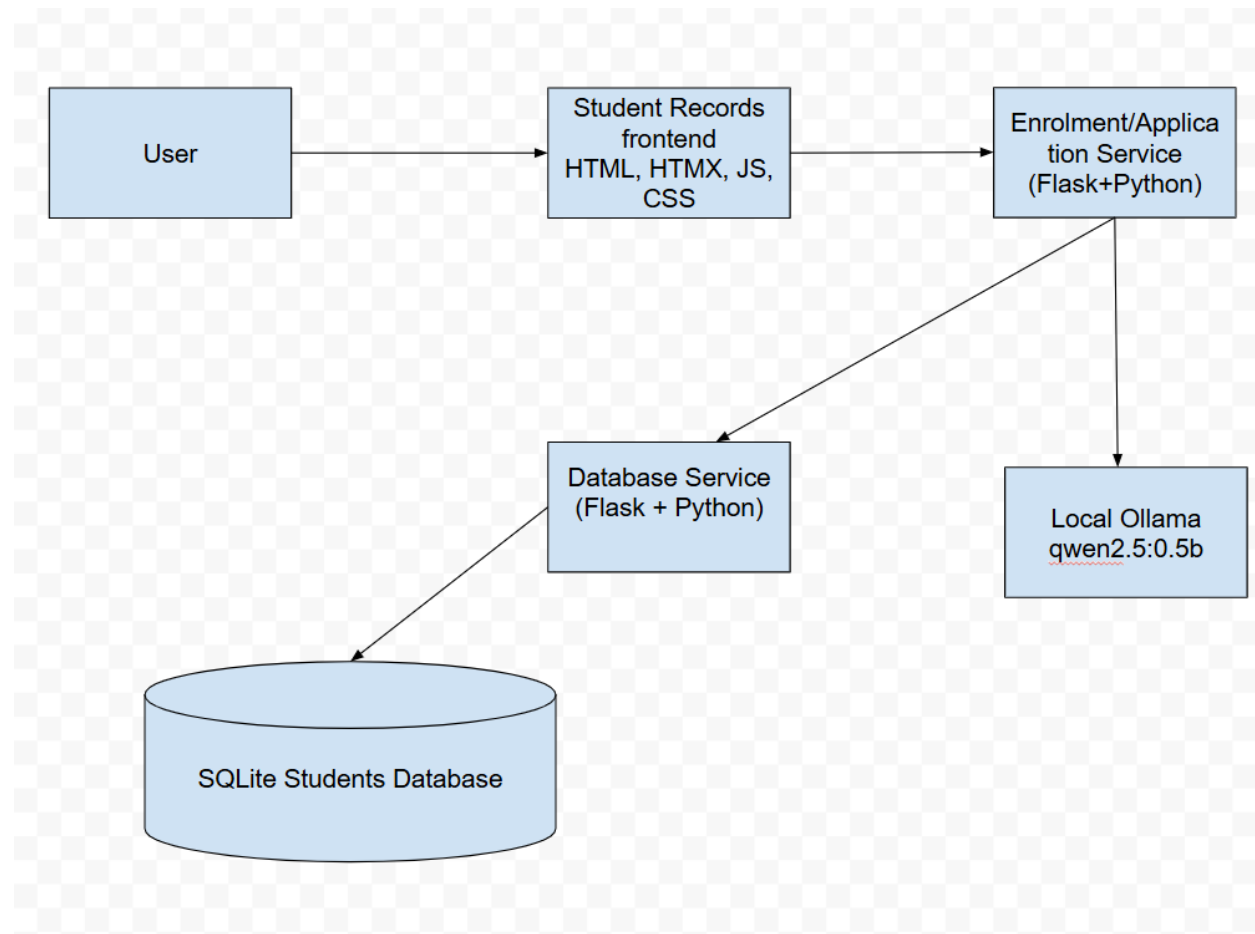
During Release 0, the team encountered and resolved two structural issues through this branching workflow:

- Inconsistent top-level naming (student1, Student2, Student4, student-5, student-3) was standardised to student-1 through student-5 in a coordinated update to main
- A real merge conflict occurred in the shared docker-compose.yml when two students' service additions landed close together; it was resolved by merging both services' blocks rather than discarding either side's work

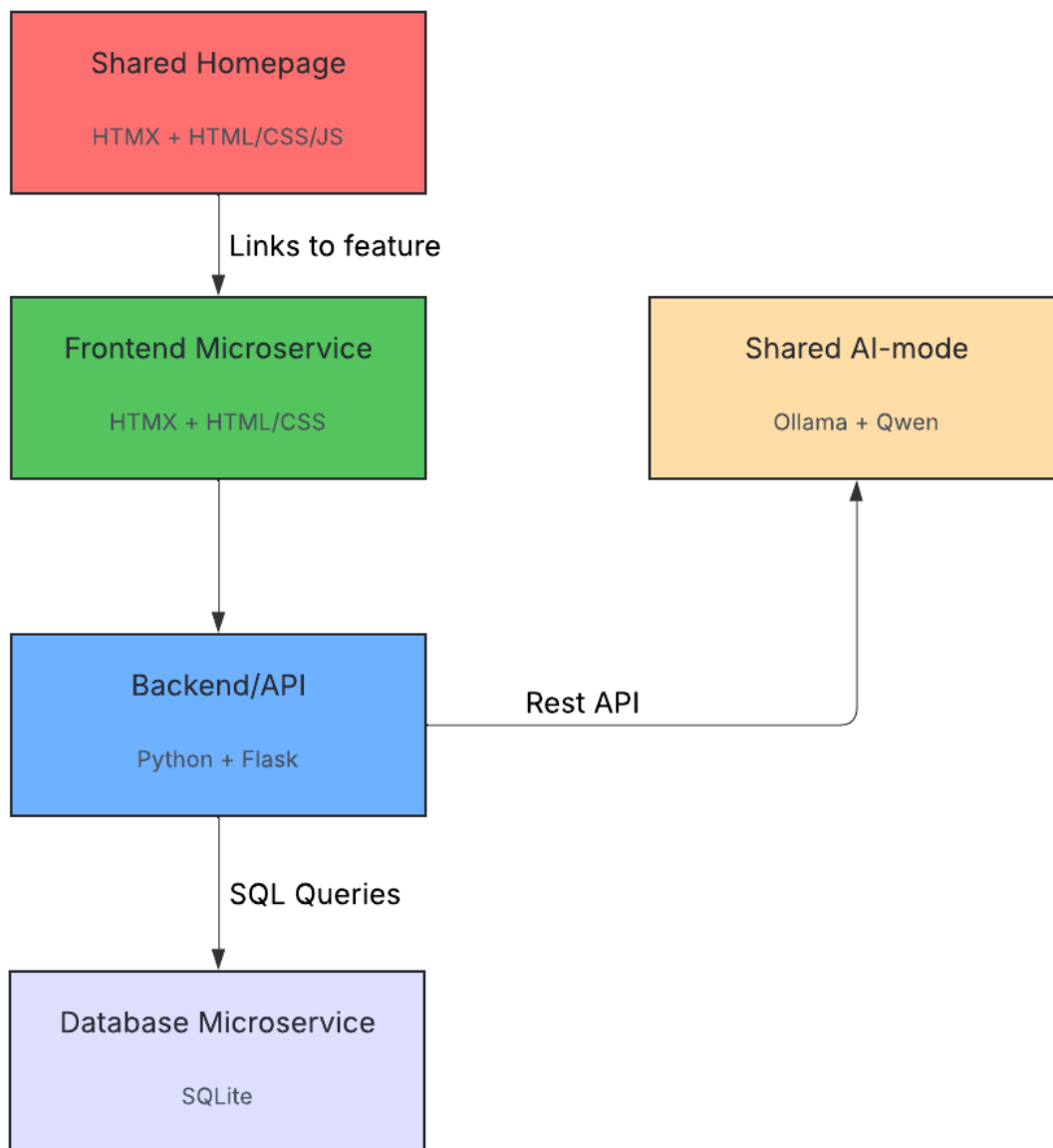
Commits are made in small, regular increments and pushed early to reduce the size and difficulty of merge conflicts, consistent with the mitigation strategy documented in each student's Risk Management Plan (Section 2.4).

4.0 Individual Software Architecture Diagrams (Frontend, Backend/API, Database)

4.1 Student 1



4.2 Student 2 – Jeriko Arceo



4.3 Student 3 – Lazizbek Ismoilov

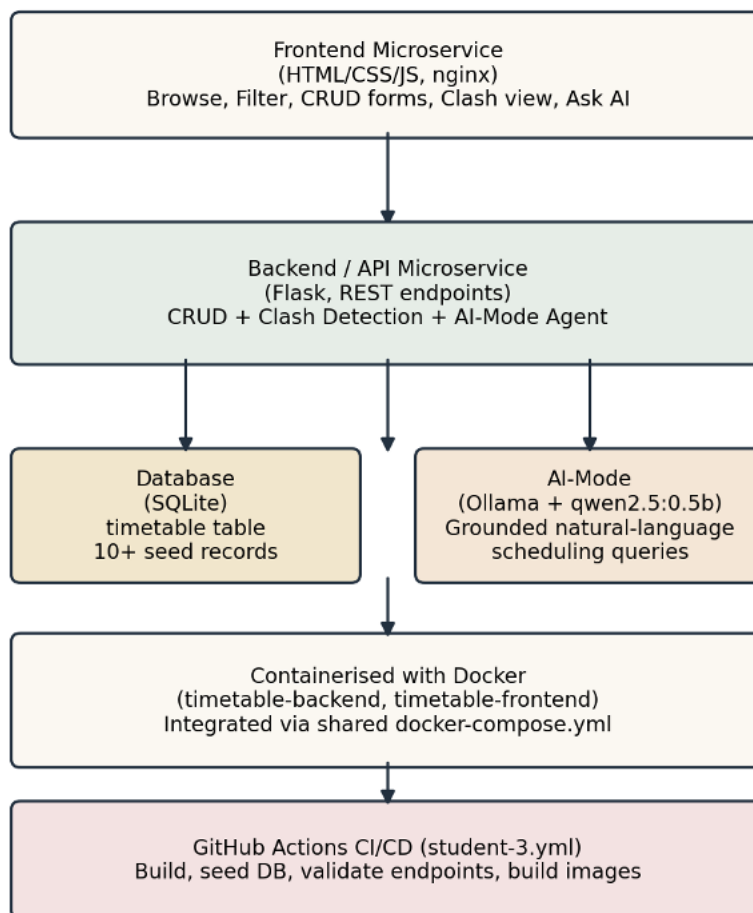
The Timetable & Class Scheduling feature uses a microservice architecture consisting of a Frontend Microservice, Backend/API Microservice, SQLite Database, and a local AI-Mode agent.

The Frontend Microservice (served via nginx) provides the Browse, Manage, Clashes, and Ask AI views, with role-based visibility for Student, Lecturer, and Admin.

The Backend/API Microservice is implemented with Flask and exposes REST endpoints for CRUD operations, clash detection, and the AI-mode agent. It owns the SQLite timetable table exclusively — no other student's service reads or writes it directly.

For AI-mode, the backend retrieves live timetable data and injects it into the Ollama (qwen2.5:0.5b) prompt as grounding context before answering natural-language scheduling questions.

Timetable & Class Scheduling — Microservice Architecture



4.4 Student 4 Yi Zhang

The Course and Enrollment Management feature uses a microservice architecture.

The main components are the Frontend Service, Enrollment Service, Database Service and local Ollama AI service.

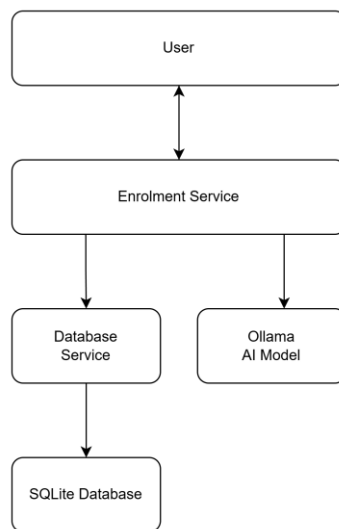
The Frontend Service provides the user interface for Course Management, Student Enrolment and AI Mode.

The Enrolment Service is implemented with Flask and acts as the application layer. It receives requests from the frontend and communicates with the Database Service.

The Database Service is also implemented using Flask and manages the SQLite database containing course and enrolment information.

For AI Mode, the Enrolment Service communicates with the local Ollama service.

The basic architecture is:



This structure separates the user interface, application logic, database operations and AI functionality.

It also makes the Course and Enrollment Management feature easier to integrate with the other services in the final group application.

4.5 Student 5

The Assessment & Grades Management feature uses a three-service microservice architecture consisting of a Frontend Service, Assessment Service, and Assessment Database Service.

The **Frontend Service** provides the user interface for managing assessments and grades. It is implemented using HTML, CSS, and JavaScript and is served through Nginx. The frontend is available on port **8085** and communicates with the Assessment Service through REST API requests.

The **Assessment Service** is implemented using Flask and acts as the main application layer. It provides the assessment and grade CRUD endpoints, input validation, cross-service integration, error handling, and AI Mode. The service runs on port **5021**.

The **Assessment Database Service** is implemented using Flask and SQLite and runs on port **5022**. It stores the assessment and grade records and provides database operations to the Assessment Service. A named Docker volume, `assessment_grades_data`, is used to persist the SQLite data.

The Assessment Service also communicates with the **Student Records service (Student 1)** to validate student IDs and retrieve student information. It communicates with the **Course Management service (Student 4)** to retrieve course information.

For AI Mode, the Assessment Service communicates with the local **Ollama** service using the `qwen2.5:0.5b` model. Relevant assessment and grade information is retrieved before the AI response is generated.

The basic architecture is:

User → Assessment & Grades Frontend → Assessment Service → Assessment Database Service

The Assessment Service also communicates with:

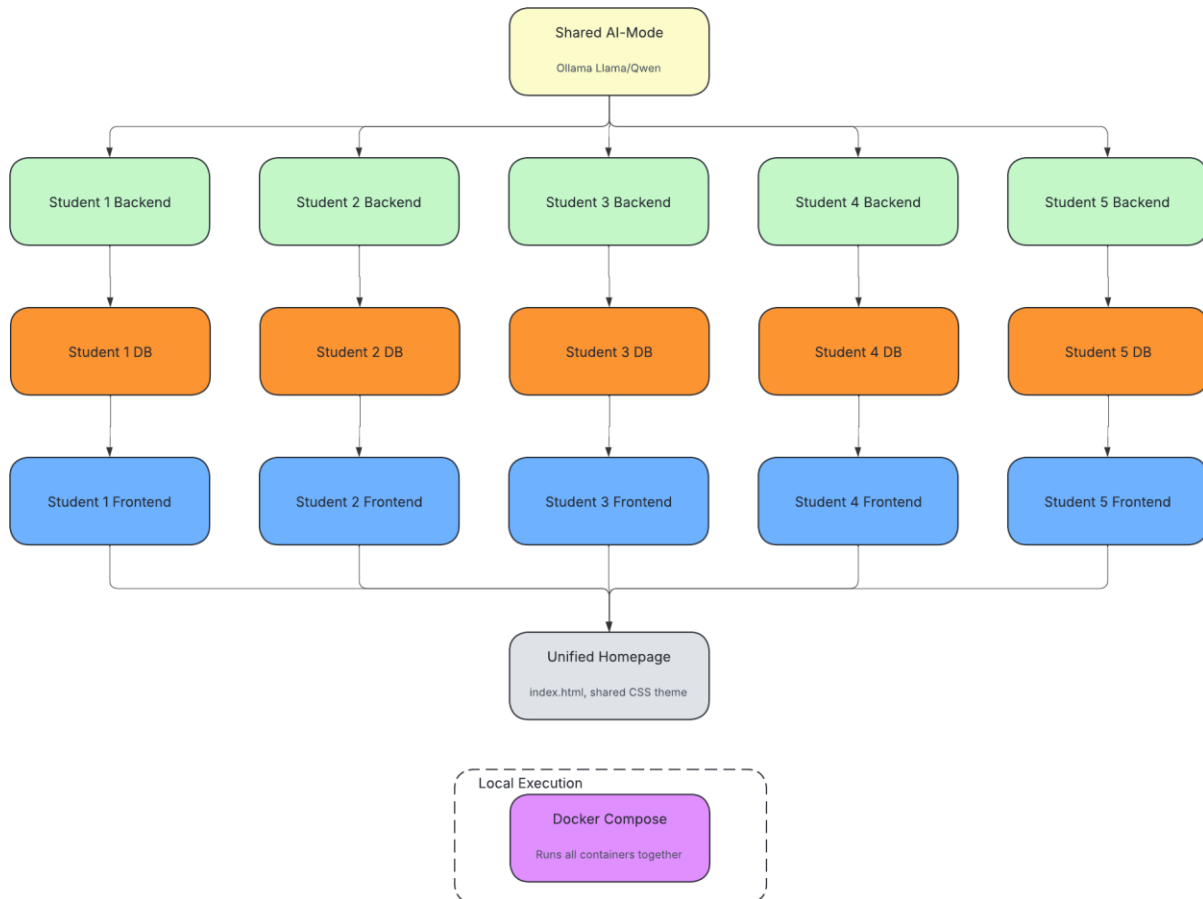
Assessment Service → Student 1 Student Records Service

Assessment Service → Student 4 Course Management Service

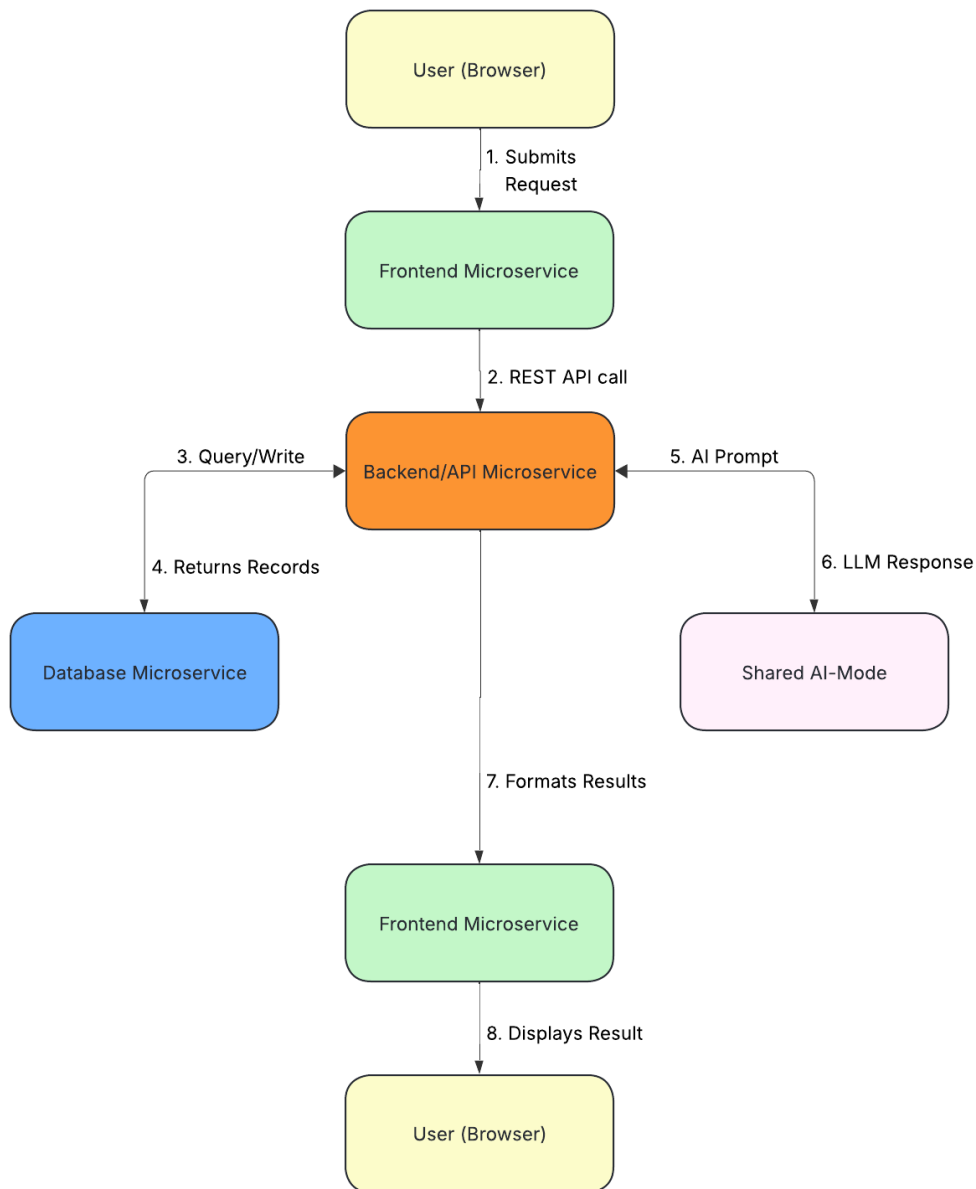
Assessment Service → Ollama AI Service

5.0 Integrated Software Architecture

5.1 Release 0 Integrated Architecture Diagram



5.2 Component Interaction Description

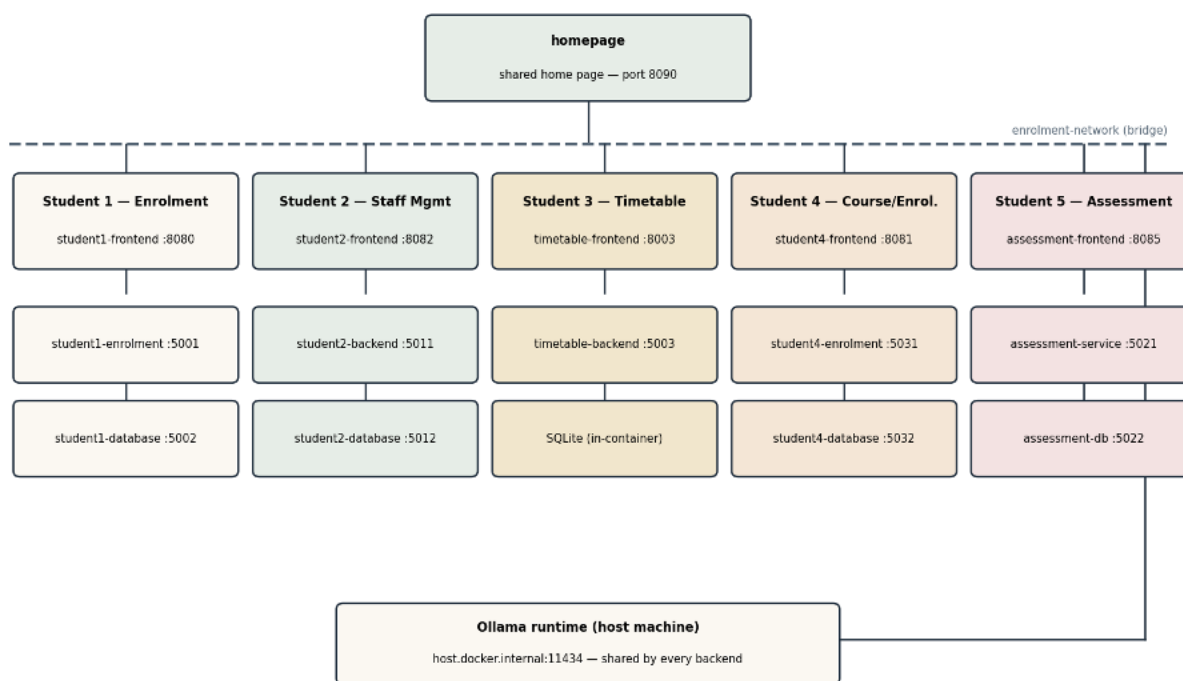


6.0 Docker Compose Architecture

6.1 Docker Compose Architecture Diagram

The full application runs as 15 containers defined in a single shared docker-compose.yml at the repository root, all connected via one Docker bridge network (enrolment-network) so any service can reach any other by its service name.

Docker Compose Architecture — Group 18 Integrated Application (15 services)



6.2 Service/Container Descriptions

All 15 services, their role, and their externally published port:

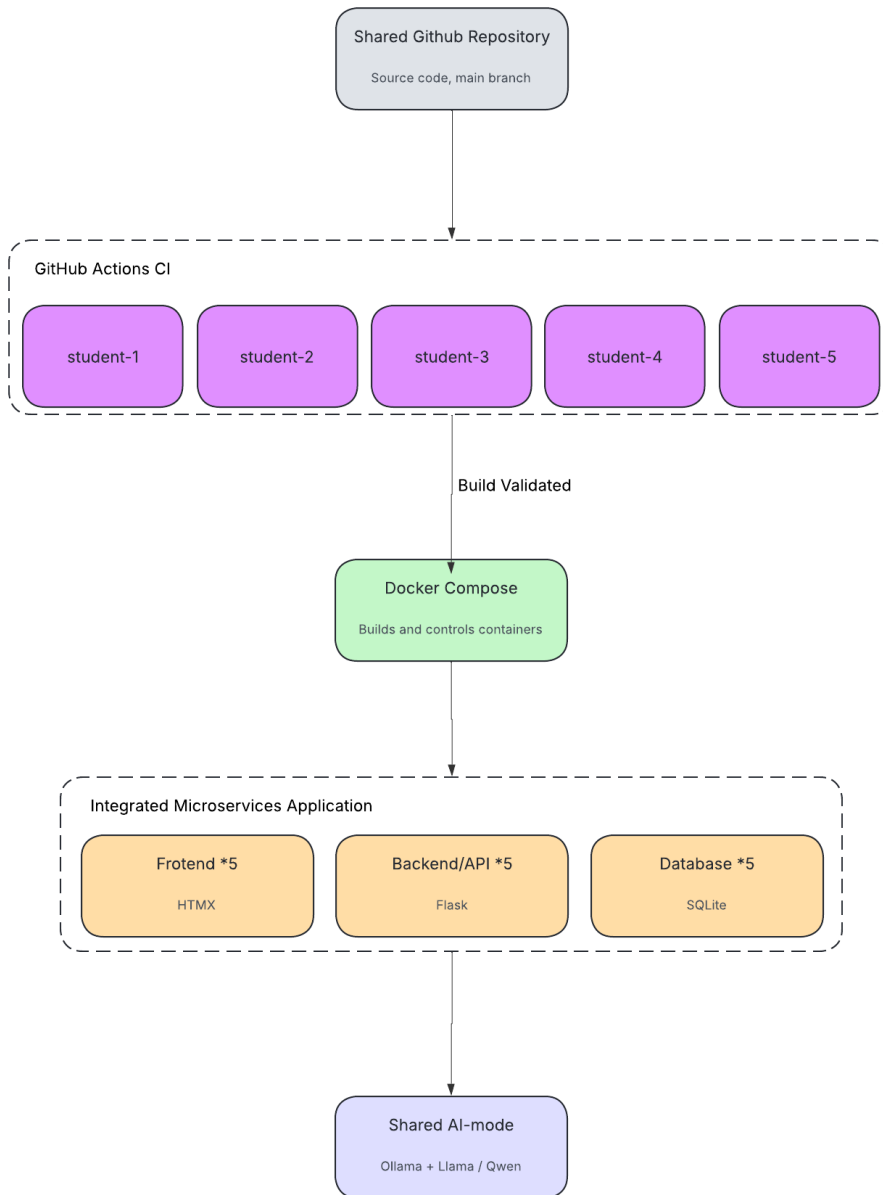
Service (container name)	Purpose & Port
homepage	Shared home page (nginx) linking to all five features — port 8090
student1-frontend	Student 1 (Enrolment) frontend — port 8080
student1-enrolment	Student 1 backend/API (Flask) — port 5001
student1-database	Student 1 database service — port 5002
student2-frontend	Student 2 (Staff Management) frontend — port 8082
student2-backend	Student 2 backend/API (Flask) — port 5011
student2-database	Student 2 database service — port 5012
timetable-frontend	Student 3 (Timetable & Class Scheduling) frontend — port 8003
timetable-backend	Student 3 backend/API (Flask), owns the SQLite timetable table directly (no separate database container) — port 5003
student4-frontend	Student 4 (Course/Enrolment) frontend — port 8081
student4-enrolment	Student 4 backend/API (Flask) — port 5031

student4-database	Student 4 database service — port 5032
assessment-frontend-service	Student 5 (Assessment/Grades) frontend — port 8085
assessment-service	Student 5 backend/API (Flask) — port 5021
assessment-database-service	Student 5 database service — port 5022

Every backend service shares the same `host.docker.internal:11434` connection to reach Ollama, which runs on the host machine rather than inside a container. Each feature's database uses a named Docker volume for persistence (e.g. `student1_database_data`, `assessment_grades_data`) so data survives container restarts. Getting all 15 services running together required resolving three integration issues: inconsistent folder naming across students, `docker-compose.yml` paths left pointing at old pre-rename folder locations, and container name conflicts between standalone per-feature testing and the shared stack.

7.0 DevOps Pipeline Architecture

7.1 Overall DevOps Architecture Diagram (GitHub, Actions, Compose, microservices, AI-Mode)



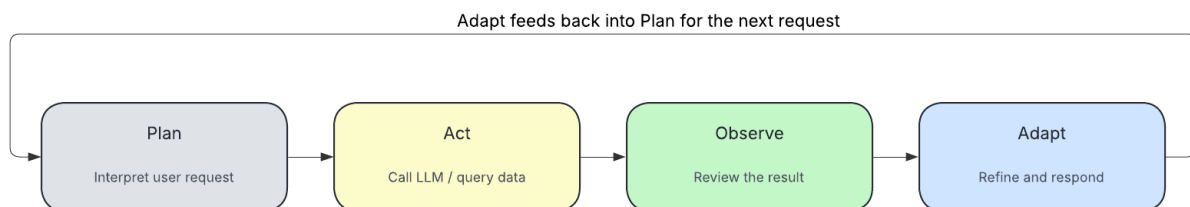
7.2 Pipeline Stage Descriptions

Stage	Description
1. Source code commit	Team members commit and push code changes to their individual feature branches, then merge into the shared GitHub repository via pull requests. This is the entry point that triggers the pipeline.
2. GitHub Actions CI	Each push or pull request affecting a student's designated folder (student-x/**) triggers that student's individual workflow file (student-1.yml to

	student-5.yml). Each workflow independently checks out the code, sets up the required runtime (e.g. Python), installs dependencies, validates syntax, and builds Docker images for that student's frontend, backend/API, and database microservices.
3. Build validation	Each workflow confirms that its microservices build successfully as Docker images and that the code is free of syntax errors before allowing changes to be merged into the main branch. This acts as a quality gate, catching integration-breaking issues early and per-student rather than at the end of the project.
4. Docker Compose build and orchestration	Once individual microservices are validated, the shared docker-compose.yml file builds and starts all five students' containers together as a single multi-container application, wiring up networking between each frontend, backend, and database set.
5. Integrated microservices application	All five frontend, backend/API, and database microservice sets run together, communicating via REST APIs, and are linked through the unified home page (index.html) with a shared CSS theme.
6. AI-Mode integration	Each backend/API microservice connects to the shared AI-Mode service (Ollama runtime with an approved open-source LLM — Llama, Qwen, or DeepSeek), enabling AI-assisted functionality across all microservices as part of the Plan → Act → Observe → Adapt workflow.
7. Local execution and validation	The integrated application is run and manually tested locally via docker compose up, with evidence (logs, screenshots) captured to demonstrate successful startup, container health, and functional CRUD/AI operations.

8.0 Agentic AI Workflow

8.1 Plan → Act → Observe → Adapt diagram



8.2 Workflow explanation and role of AI agents

Stage	Explanation
Plan	When the user submits a request (natural language query or CRUD action) through any frontend microservice, the backend/API microservice will interpret the request and determine the best course of action, whether to connect to the database service and make a direct query or to call upon the AI-mode service.
Act	The backend/API executes the planned action: sending the request to the shared AI-Mode service (Ollama running an approved LLM), and/or performing the relevant database operation (Create, Read, Update, Delete).
Observe	The system evaluates the result returned from the LLM or database - checking whether the response is complete, relevant, and correctly formatted before it is passed back to the user.
Adapt	The result is formatted for display in the frontend. If the response is incomplete, ambiguous, or an error occurred, the system adjusts for example, retrying the query, narrowing the AI prompt, or returning a clarifying message to the user. The cycle then restarts with the next user interaction.

9.0 GitHub Actions Workflows

9.1 Description of student-1.yml - Rigel Rivamonte

The purpose of this workflow is to automate the validation and Docker image builds for the Student Records feature in Student 1. It is designed to run only when relevant Student 1 files or the workflow itself are changed, helping to confirm that the feature is functioning correctly before it is merged into the shared project.

Triggers:

- Push: runs on pushes to the rigel or main branches when files inside student-1/ or the workflow file are changed
- Pull request: runs when a pull request targets main and the changed files are within student-1/

The workflow performs the following steps:

- Checks out the repository
- Sets up Python 3.11
- Installs the Student 1 dependencies
- Runs the Student Records unit tests

- Validates Python syntax for the database application, database initialisation, Flask application, normal UI routes, AI routes, database API service, LLM client, prompt loader, and HTML formatters
- Builds the frontend Docker image
- Builds the enrolment Docker image
- Builds the database Docker image

When all checks pass, this workflow confirms that the Student 1 feature is functionally valid, syntactically correct, and ready to be containerised before it is integrated into the wider application.

9.2 Description of student-2.yml - Jeriko Arceo

The purpose is to automate the build and validation of the staff management feature which only triggers on relevant code changes.

Triggers:

- Push – runs when code changes are pushed to the following branches “jeriko” and “jeriko-workingAttempt”, only when files inside the “student-2” folder are changed
- Pull – runs when a pull request targets the main branch but only if files in “student-2” are modified

The workflow then performs the following steps:

- Pulls the repository code into the runner
- Sets up Python 3.11
- Installs the backend dependencies
- Validates python syntax
- Builds the frontend docker image
- Builds the backend docker image
- Builds the database docker image
- Validates that the shared docker compose file is valid and resolves correctly

When all steps have run successfully, the file confirms that the three microservices build correctly and the Python code is free of syntax errors before merging into shared branches.

9.3 Description of student-3.yml

The purpose of student-3.yml is to automate the build and validation of the Timetable & Class Scheduling feature, triggering only on relevant code changes.

Triggers:

- Push — runs on pushes to main or any student-3/** branch, only when files inside the student-3/ folder or the workflow file itself are changed
- Pull request — runs when a PR targets main, only if files in student-3/ are modified

The workflow performs the following steps:

- Checks out the repository
- Sets up Python 3.12
- Installs backend dependencies
- Initialises and seeds the SQLite database
- Verifies the database was seeded with exactly 10 records
- Starts the real Flask application
- Validates that core endpoints (/timetable, /timetable/<id>, /timetable/clashes) respond successfully
- Builds the backend Docker image
- Builds the frontend Docker image

When all steps succeed, the workflow confirms the microservice builds correctly, the database seeds as expected, and the live endpoints respond — before the change is merged into the shared branch. One issue found and fixed during setup: the initial version only triggered on changes under student-3/**, so pushing the workflow file itself did not trigger a run — fixed by adding the workflow file's own path to the trigger filter.

9.4 Description of student-4.yml - Yi Zhang

Student 4 uses the YiZhang.yml workflow for the Course and Enrollment Management feature.

The workflow can be manually started and is also triggered when changes are pushed to the Yi-Zhang branch or when a pull request affecting the Student 4 files is created.

The workflow performs the following steps:

- Checks out the repository.
- Sets up Python 3.11.
- Installs the Course and Enrollment service dependencies.
- Checks the syntax of the database service and enrolment service Python files.
- Builds the frontend Docker image.
- Builds the enrolment service Docker image.
- Builds the database service Docker image.
- Validates the Student 4 Docker Compose configuration.

The main purpose of this workflow is to detect Python or Docker configuration problems before the Course and Enrolment Management feature is integrated into the main application.

9.5 Description of student-5.yml

The Student 5 GitHub Actions workflow is used to automatically validate the Assessment & Grades Management feature when changes are pushed or submitted through a pull request affecting the Student 5 files.

The workflow uses Python 3.11 and performs the following steps:

- Checks out the repository.
- Sets up Python 3.11.
- Installs the required Python dependencies and testing packages.
- Builds the Assessment Database Service Docker image.
- Builds the Assessment Service Docker image.
- Builds the Assessment Frontend Service Docker image.
- Creates the required Docker network.
- Starts the database and backend services.
- Performs smoke tests against the database and Assessment API.
- Runs the automated Student 5 test suite.
- Cleans up the test containers after validation.

The main purpose of the workflow is to identify Python, API, testing, and Docker problems before the Assessment & Grades Management feature is integrated into the main application.

10.0 Implementation Summary

10.1 Release 0 scope and completed features

Release 0 focused on building the main University Management System using independent microservices. Each feature was developed with separate frontend, backend/API and database components, and the services were integrated through the shared docker-compose.yml.

The completed features include:

1. Student Records Management – student CRUD, search, filtering and AI Mode.
2. Staff Management – staff CRUD, staff-course assignments and AI-assisted queries.
3. Timetable and Class Scheduling – timetable CRUD, clash detection, role-based views and AI Mode.
4. Course and Enrolment Management – course and enrolment CRUD, Student Records API integration and AI Mode.
5. Assessment and Grades – included as a separate feature in the integrated microservice structure.

Docker, Ollama AI Mode and GitHub Actions were also used to support integration and automated validation. The Release 0 plan required the individual CRUD features, AI integration and the integrated Docker Compose application.

10.2 Deviations from Plan

Most of the planned Release 0 functions were completed. However, some changes were made during development and integration.

Some cross-feature integration was simplified or left for future releases. Technical changes were also required to resolve issues with Docker configuration, service communication, CORS, and shared project files. These changes mainly affected the implementation process and did not change the main objectives of Release 0.

11.0 GitHub Actions Evidence (i)

[Jeriko Arceo](#)

jeri0205 merged commit a371e3a into main 2 days ago [Hide details](#) [Revert](#)

4 checks passed

✓	build-and-validate	Details
✓	build-and-validate	Details
✓	build-and-validate	Details
✓	build-and-validate	Details

Lazizbek Ismoilov

← Student 3 - Timetable & Class Scheduling CI

✓ Merge branch 'main' of <https://github.com/rigels51/ASD-Project-Group18> #9 [Re-run all jobs](#) [...](#)

Summary

All jobs [⌵](#)

✓ build-and-validate

Run details

🕒 Usage

📄 Workflow file

Triggered via push 35 minutes ago

LazIsmoilov pushed [84834f2](#) [main](#)

Status **Success**

Total duration **30s**

Artifacts **-**

student-3.yml
on: push

✓ build-and-validate 26s

[🔗](#) [-](#) [+](#)

The workflow validated: dependency installation, database seeding (10 records confirmed), live endpoint checks against /timetable, /timetable/<id>, and /timetable/clashes, and successful builds of both the backend and frontend Docker images.

Rigel Rivamonte

ci: fix Rigel workflow validation5f9b65a

Merge remote-tracking branch 'origin/rigel' into rigel856f260

All checks have passed

2 successful checks

Rigel Rivamonte - Student Records / build-and-validate (pull_request)

Successful in 32s

...

Rigel Rivamonte - Student Records / build-and-validate (push)

Successful in 27s

...

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

← Rigel Rivamonte - Student Records

✓ Merge pull request #24 from rigels51/rigel #14

Summary

All jobs

✓ build-and-validate

Run details

Usage

Workflow file

Triggered via push now

rigels51 pushed → d9949d4 main

Status: **Success**

Total duration: **40s**

Artifacts: —

RigelRivamonte.yml

on: push

✓ build-and-validate 37s

Yi Zhang

← Yi Zhang - Course and Enrollment

✓ Update app.py #15 Re-run all jobs

Summary

All jobs

✓ build-and-validate

Run details

Usage

Workflow file

Triggered via push yesterday

YiZhang2004 pushed → 9de602f Yi-Zhang

Status: **Success**

Total duration: **29s**

Artifacts: —

YiZhang.yml

on: push

✓ build-and-validate 26s

Annotations

1 warning

12.0 Docker Compose Evidence

Lazizbek Ismoilov — Timetable & Class Scheduling

Both services (timetable-backend on port 5003, timetable-frontend on port 8003) run successfully standalone and as part of the shared root docker-compose.yml alongside all other student services, confirmed via a full 15-service docker compose up --build run.

```
: 1048576:1048576
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker processes
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-frontend | 2026/09/08 04:13:25 [notice] 1#1: start worker process 3
timetable-backend | * Serving Flask app 'app'
timetable-backend | * Debug mode: on
timetable-backend | WARNING: This is a development server. Do not use it in
production deployment. Use a production WSGI server instead.
timetable-backend | * Running on all addresses (0.0.0.0)
timetable-backend | * Running on http://127.0.0.1:5003
timetable-backend | * Running on http://172.21.0.2:5003
timetable-backend | Press CTRL+C to quit
timetable-backend | * Restarting with stat
timetable-backend | * Debugger is active!
timetable-backend | * Debugger PIN: 540-282-993
```

STUDENT 3

Timetable & Class Scheduling

University Management System — Group 18

Home

2026-S2

VIEWING AS

Student

Lecturer

Admin

SECTIONS

Browse

Ask AI

Filter by course code, e.g. ASD101

Filter

View all sessions

ID	COURSE	TYPE	DAY	START	END	ROOM	SEMESTER
1	ASD101	Lecture	Monday	09:00	11:00	CB01.02.15	2026-S2
2	ASD101	Tutorial	Monday	11:00	12:00	CB01.02.16	2026-S2
3	WEB201	Lecture	Tuesday	09:00	11:00	CB01.03.01	2026-S2
4	WEB201	Lab	Tuesday	13:00	15:00	CB01.03.02	2026-S2
5	DBS101	Lecture	Wednesday	10:00	12:00	CB02.01.10	2026-S2
6	DBS101	Tutorial	Wednesday	13:00	14:00	CB02.01.11	2026-S2
7	NET201	Lecture	Thursday	09:00	11:00	CB02.02.05	2026-S2
8	NET201	Lab	Thursday	14:00	16:00	CB02.02.06	2026-S2
9	SEC301	Lecture	Friday	09:00	11:00	CB03.01.01	2026-S2
10	SEC301	Tutorial	Friday	11:00	12:00	CB03.01.02	2026-S2

```

#127 [student1-database] resolving provenance for metadata file
#137 DONE 0.1s
[+] up 32/32
✓ Image asd-project-group18-student4-frontend Built 8.7s
✓ Image asd-project-group18-student2-database Built 8.7s
✓ Image asd-project-group18-student4-database Built 8.7s
✓ Image asd-project-group18-assessment-frontend-service Built 8.7s
✓ Image asd-project-group18-student2-frontend Built 8.7s
✓ Image asd-project-group18-assessment-database-service Built 8.7s
✓ Image asd-project-group18-homepage Built 8.7s
✓ Image asd-project-group18-student4-enrolment Built 8.7s
✓ Image asd-project-group18-student1-enrolment Built 8.7s
✓ Image asd-project-group18-student2-backend Built 8.7s
✓ Image asd-project-group18-timetable-backend Built 8.7s
✓ Image asd-project-group18-timetable-frontend Built 8.7s
✓ Image asd-project-group18-student1-database Built 8.7s
✓ Image asd-project-group18-student1-frontend Built 8.7s
✓ Image asd-project-group18-assessment-service Built 8.7s
✓ Network asd-project-group18_enrolment-network Created 0.0s
✓ Network asd-project-group18_default Created 0.0s
✓ Container student2-database Started 1.4s
✓ Container assessment-database-service Started 1.3s
✓ Container timetable-backend Started 1.3s
✓ Container database-service Started 1.4s
✓ Container group18-homepage Started 1.2s
✓ Container student4-database Started 1.2s
✓ Container student4-enrolment Started 1.7s
✓ Container enrolment-service Started 1.8s
✓ Container student2-backend Started 1.8s
✓ Container assessment-service Started 1.4s
✓ Container timetable-frontend Started 1.5s
✓ Container student1-frontend Started 2.1s
... 3 more
PS C:\Users\zy110\Desktop\ASD-Project-Group18>

```

Rigel Rivamonte ---- Student Records Management

```

#30 [student1-database] exporting to image
#30 exporting layers done
#30 exporting manifest sha256:e0ffdf15739c9b51f20b825da20eb9d206b7ddcee75a4cd55cc3e164a4276b4d done
#30 exporting config sha256:8ad04b3da47ce03c5f811ae3b483d2cd5ad28477ca1a2a02bd912142caa1545c done
#30 ...

#31 [student1-enrolment] exporting to image
#31 exporting layers done
#31 exporting manifest sha256:0bc6110d896a77b4576e4b2267991fef037f684c0b266e44b85a10306f1a6de0 done
#31 exporting config sha256:b5c346ee92802e0bdb5161988145f00d721f41260d71107ccc19afc73e85943 done
#31 exporting attestation manifest sha256:5c1ef8ee7e742700ca73065d3acdafbe56a8c7ea0ec7cb1d3224d29871a851e0 0.0s done
#31 exporting manifest list sha256:74101fd75e59375cd18b72df65e7cbb8912c9330f3ee7f674a5d6198e838d4dc 0.0s done
#31 naming to docker.io/library/asd-project-group18-student1-enrolment:latest done
#31 unpacking to docker.io/library/asd-project-group18-student1-enrolment:latest 0.0s done
#31 DONE 0.1s

#30 [student1-database] exporting to image
#30 exporting attestation manifest sha256:286fb9edd5416641e35930bd8f0dd5e4ed627a27c6f83b4008b209cff2f791af 0.0s done
#30 exporting manifest list sha256:62ef7007238e1be16a0956129a8a5272c38bfca1e3d375ea1289106e30774be2 0.0s done
#30 naming to docker.io/library/asd-project-group18-student1-database:latest done
#30 unpacking to docker.io/library/asd-project-group18-student1-database:latest 0.0s done
#30 DONE 0.1s

#32 [student1-enrolment] resolving provenance for metadata file
#32 DONE 0.1s

#33 [student1-database] resolving provenance for metadata file
#33 DONE 0.0s
[+] up 6/6
✓ Image asd-project-group18-student1-frontend Built 2.1s
✓ Image asd-project-group18-student1-database Built 2.1s
✓ Image asd-project-group18-student1-enrolment Built 2.1s
✓ Container student1-frontend Running 0.0s
✓ Container student1-enrolment Running 0.0s
✓ Container student1-database Running 0.0s
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18>

```

13.0 Agentic Loop Workflow Record

```
=====
COURSE & ENROLLMENT MANAGEMENT AGENTIC AI LOOP
=====

PLAN
Validate Course and Enrollment Management microservices.
{'goal': 'Validate Course and Enrollment Management using a local Agentic AI workflow', 'database_plan': ['Check that at least 10 course records exist', 'Check that at least 10 enrolment records exist', 'Validate required course fields', 'Validate required enrolment fields'], 'endpoint_plan': ['GET /courses', 'GET /enrolments', 'POST /ask', 'POST /ask-with-context']}

ACT
Run database, API, availability and AI validation checks.

OBSERVE: Database Check
  Courses found: 10
  Course ASD101 -> OK
  Course DBS101 -> OK
  Course WEB201 -> OK
  Course NET201 -> OK
  Course SEC301 -> OK
  Course DAT201 -> OK
  Course AIT101 -> OK
  Course CLD201 -> OK
  Course SWE301 -> OK
  Course PMT101 -> OK
  Enrolments found: 10
  Enrolment 1 -> OK
  Enrolment 2 -> OK
  Enrolment 3 -> OK
  Enrolment 4 -> OK
  Enrolment 5 -> OK
```

```
  Course WEB201 -> OK
  Course NET201 -> OK
  Course SEC301 -> OK
  Course DAT201 -> OK
  Course AIT101 -> OK
  Course CLD201 -> OK
  Course SWE301 -> OK
  Course PMT101 -> OK
  Enrolments found: 10
  Enrolment 1 -> OK
  Enrolment 2 -> OK
  Enrolment 3 -> OK
  Enrolment 4 -> OK
  Enrolment 5 -> OK
  Enrolment 6 -> OK
  Enrolment 7 -> OK
  Enrolment 8 -> OK
  Enrolment 9 -> OK
  Enrolment 10 -> OK
Course and enrolment data validation passed

OBSERVE: Course Availability
ASD101: capacity=30, active=2, available=28
DBS101: capacity=25, active=1, available=24
WEB201: capacity=40, active=1, available=39
NET201: capacity=30, active=1, available=29
SEC301: capacity=25, active=0, available=25
DAT201: capacity=35, active=1, available=34
AIT101: capacity=30, active=1, available=29
CLD201: capacity=25, active=0, available=25
SWE301: capacity=40, active=0, available=40
PMT101: capacity=30, active=0, available=30

OBSERVE: Live Endpoint Check
Checked /courses -> HTTP 200, content_ok=True
Checked /enrolments -> HTTP 200, content_ok=True
Checked /ask -> HTTP 200, content_ok=True
Checked /ask-with-context -> HTTP 200, content_ok=True
```

```
Checked /ask-with-context -> HTTP 200, content_ok=True

IMPLEMENTATION AGENT
Model: qwen2.5:0.5b

No evidence-backed improvement identified.

REVIEW AGENT
Model: llama3.1:8b

Risk: Validation for /students endpoint is missing.
Correction: Add validation for /students endpoint to the evidence list.
Retest: Validate /students endpoint after implementation.

HUMAN DECISION

HUMAN REVIEW
1 - Accept
2 - Partially Accept
3 - Reject
Decision: 1

Decision: Accept

ADAPT: Apply recommendation and rerun validation.

LOOP COMPLETE
PS C:\Users\zy110\Desktop\ASD-Project-Group18> █
```

13.1 Shared Agentic Loop Development Contributions

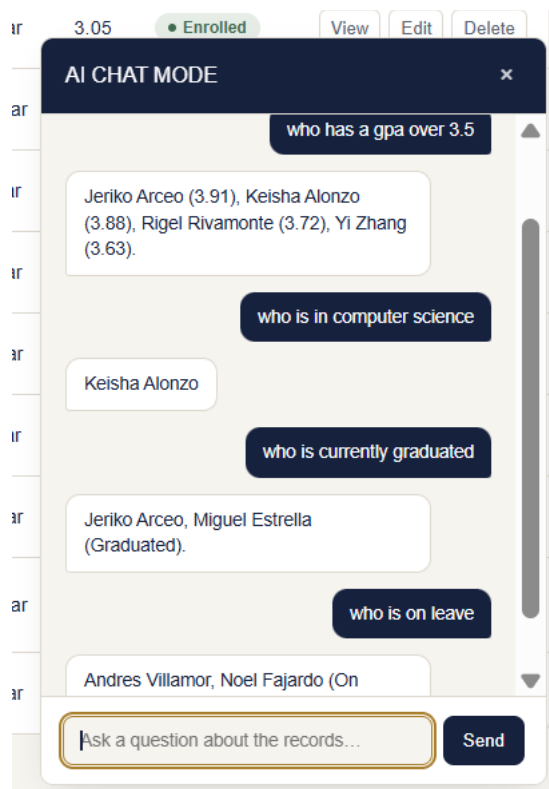
A shared Agentic Loop was added at the project level using `agentic_loop/main.py`. It provides a common entry point for running feature-level Agentic Loops.

The shared loop currently runs the Course and Enrollment Management Agentic Loop. It checks database records, API endpoints, AI responses, and review results.

The workflow follows the Plan → Act → Observe → Review → Human Decision → Adapt process and completes with a final validation result.

13.2 Individual Prompt Assets (i)

Rigel Rivamonte



The Student Records AI feature uses a local Ollama model, qwen2.5:0.5b. Before a context-aware AI request is sent, the application retrieves the latest student records from the database and converts them into JSON prompt context. The prompt instructs the model to:

- Use only the supplied student records.
- Treat the supplied records as the source of truth.
- Copy names, IDs, courses, statuses, and values exactly.
- Never invent, alter, or substitute student information.
- State clearly when requested information is unavailable.
- Answer briefly and directly.
- Avoid creating, updating, or deleting records through the AI response.

Example supported questions include:

- “Who is on leave?”
- “Who has a GPA above 3.5?”
- “Who is studying in computer science?”
- “How many students are in the records?”

Lazizbek Ismoilov

The AI agent for Timetable & Class Scheduling is grounded before every response: the backend first queries the live timetable table, formats each session as a plain-text line (course, type, day, time, room, semester), and injects that as context into the system prompt.

System prompt instruction: “You are a concise university timetable assistant. Only use the session data provided below to answer. If the answer isn’t in the data, say so.” followed by the live timetable data.

This prevents the small local model (qwen2.5:0.5b) from inventing sessions, rooms, or times that don’t exist — the same grounding pattern used in the Lab 4 agentic loop exercise, applied here to a live REST endpoint rather than a static file.

Example questions supported:

- When is my next ASD101 class?
- What sessions are scheduled on Monday?
- Is there a Tutorial for WEB201?

ASK THE AI AGENT

When is my next ASD101 class?

Ask

The next ASD101 class is on Monday, 09:00-11:00 in CB01.02.15 (2026-S2).

Yi Zhang

AI prompts were used to control the behaviour of the local language model in the Course and Enrollment Management feature.

For general questions, the model is instructed to act as a Course and Enrollment Management assistant and provide clear and short answers.

For the database-context AI function, stronger instructions are used.

The model is told to answer using the supplied course and enrolment information and not invent courses, students, enrolments or numerical values that are not included in the database context.

If the requested information is not available, the AI should explain that the information is not available.

Before the prompt is sent to the AI model, the application retrieves the latest course and enrolment data from the database.

The system can also calculate information such as the number of active enrolments and the number of available seats in a course.

This information is then included in the prompt together with the user's question.

For example, users can ask questions such as:

- Which courses have available seats?
- How many active enrolments are in a course?
- What course information is currently available?

This prompt design helps reduce unsupported AI responses and makes the AI function more closely connected to the actual application data.

The screenshot displays the 'AI Mode' interface with the following components:

- Header:** 'AGENTIC AI SERVICES' and 'AI Mode' with a subtitle 'Ask questions about courses and enrolments using the local AI agent.'
- Section 1: Ask Local AI Agent**
 - Subtitle: 'Ask a general question using the local Ollama model.'
 - Question input: 'Explain what the Course and Enrollment Management system does.'
 - Button: 'Ask Local Agent'
 - Response: 'The Course and Enrollment Management (C&EM) system is an educational technology platform that helps manage course offerings, student enrollment, and academic records for universities and colleges. It integrates various systems such as admissions, financial aid, registration, and grading to streamline the process of registering students for courses and managing their attendance. The system also tracks student progress through grades, assignments, and exams, providing a comprehensive view of student performance and academic progress.'
- Section 2: Ask With Database Context**
 - Subtitle: 'Ask questions using the current course and enrolment data.'
 - Question input: 'Which courses have available seats?'
 - Button: 'Ask With Context'
 - Response: 'The courses with available seats are: - Web Development (Course ID: 3) - Data Analytics (Course ID: 6) - Cyber Security (Course ID: 7)'

13.3 Review Records

Student 4 – Course and Enrollment Management

- Reviewed Course and Enrolment CRUD operations.
- Checked Docker build and Compose configuration.
- Verified Student 1 API integration using valid and invalid student IDs.
- Confirmed AI Mode uses current course and enrolment data as context.

Student 1 – Student Records Management

- Checked that all student records can be retrieved and displayed
- Verified the search by student name, ID, and course feature
- Tested student creation, update, and deletion functions
- Checked that required fields in the form return appropriate validation errors
- Verified that a missing student record returns a 404 response
- Confirmed if the AI mode uses current student records before answering contextualised questions

14.0 Known Issues and Limitations

14.1 Outstanding Issues/Bugs

Student Records Management (Student 1):

- No uniqueness check on email, so duplicate values can be stored.
- AI Mode (/ask, /ask-with-context) depends on the local Ollama service being installed and running

Timetable & Class Scheduling (Student 3):

- Lecturer “my sessions” lookup by staff_id is currently a placeholder — staff_id is unpopulated until Staff Management integration confirms a shared ID format
- Clash detection is $O(n^2)$ — fine at current scale (10s of sessions) but would need indexing at real university scale

Course and Enrollment Management (Student 4):

- Student enrolment validation depends on the Student 1 API being available.
- AI Mode depends on the local Ollama service.
- Large-scale performance has not been tested because Release 0 uses a small local dataset.

Assessment & Grades Management (Student 5):

- Grade creation and update depend on the Student 1 Student Records service being available for student ID validation.
- Course information enrichment depends on the Student 4 Course Management service.
- AI Mode depends on the local Ollama service being installed and running.
- If an external service is unavailable, some cross-feature information may not be displayed even though the core Assessment & Grades service remains available.

14.2 Technical Debt and Limitations

Student Records Management (Student 1):

- There is limited support for audit history or traceability of record changes
- The AI mode is designed for demonstration, so it doesn't yet include proper validation or fallback behaviour for unexpected inputs or unrecognisable prompts

Timetable & Class Scheduling (Student 3):

- Role-based access is enforced via client-side view toggling, not server-side authentication — acceptable for Release 0 scope since the spec does not require login, but would need a real auth layer for production use
- `course_code` and `staff_id` fields are ready for cross-service integration but not yet actively validated against Course Catalogue or Staff Management's live data

Course and Enrollment Management (Student 4):

- SQLite is suitable for Release 0 but may not be suitable for large production workloads.
- Student validation currently uses a direct REST API call to Student 1 without caching.
- Duplicate enrolment and course-capacity validation could be improved in future releases.

Assessment & Grades Management (Student 5):

- SQLite is suitable for Release 0 but may not be appropriate for a large production university system.
- Student and course information is retrieved through direct REST API calls without caching.
- AI Mode is read-only and is designed to answer questions using retrieved assessment and grade information rather than perform write operations.

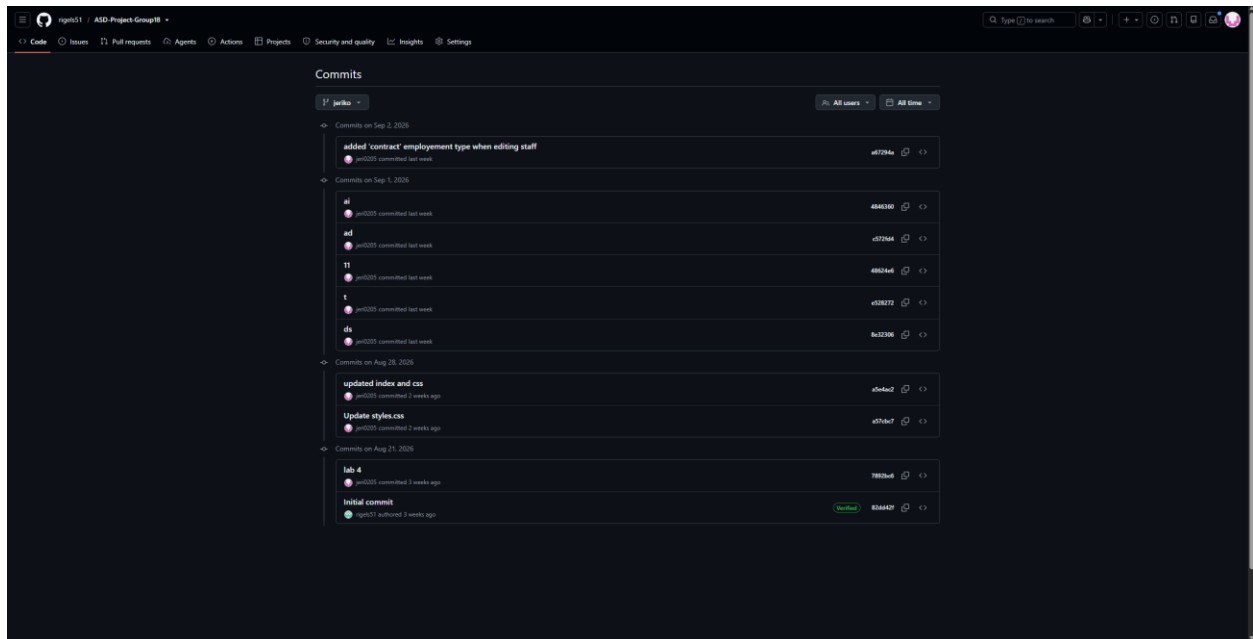
15.0 GitHub Commit Logs (i)

Rigel Rivamonte

Commits		
main	rigels51	All time
Commits on Sep 8, 2026		
Merge pull request #18 from rigels51/Student1	Verified c5e850e	<>
rigels51 authored 23 minutes ago		
test: trigger RigelRivamonte workflow	783abbb	<>
rigels51 committed 24 minutes ago		
fixing prefixes	37787ca	<>
rigels51 committed 1 hour ago		
Commits on Sep 6, 2026		
small front	dde1334	<>
rigels51 committed 2 days ago		
Merge branch 'main' of https://github.com/rigels51/ASD-Project-Group18	8380fff	<>
rigels51 committed 2 days ago		
Merge pull request #13 from rigels51/rigel	Verified d28f5fb	<>
rigels51 authored 2 days ago		
Update RigelRivamonte.yml	cc87f66	<>
rigels51 committed 2 days ago		
Commits on Sep 4, 2026		
small change	a12a675	<>
rigels51 committed 5 days ago		
small change	ca2f054	<>
rigels51 committed 5 days ago - X 0 / 1		
small frontend changes	3c7e234	<>
rigels51 committed 5 days ago		
small frontend changes	5e0e828	<>
rigels51 committed 5 days ago		
fixing yaml capitals rq	6230f07	<>
rigels51 committed 5 days ago		
fixing numbers and names in database	b64fe5e	<>
rigels51 committed 5 days ago		
fixing numbers and names in database	f966f58	<>
rigels51 committed 5 days ago		

rigels51 committed 5 days ago	
ai incorrect messages rigels51 committed 5 days ago	f137f39
Commits on Sep 3, 2026	
Update student records and AI queries rigels51 committed 5 days ago	56a958c
Merge branch 'rigel' rigels51 committed 5 days ago	8d89b80
got rid of changing name, course and id rigels51 committed 5 days ago	77d2b6f
Merge branch 'main' of https://github.com/rigels51/ASD-Project-Group18 rigels51 committed 5 days ago	3d8b6c1
Merge branch 'rigel' of https://github.com/rigels51/ASD-Project-Group18 into rigel rigels51 committed 5 days ago	9c1be55
adding our names for fun rigels51 committed 5 days ago	509d0e3
Merge pull request #11 from rigels51/rigel rigels51 authored 5 days ago	Verified 531e016
Merge branch 'main' into rigel rigels51 authored 5 days ago · 0 / 2	Verified f7c4b15
fix ai mode and ui rigels51 committed 5 days ago	6d2a422
Rename student1/database-service/app.py to Student1 rigels51 authored 5 days ago	Verified 3376005
Update workflow name for clarity rigels51 authored 5 days ago	Verified 7e48423
Rename student1.yml to RigelRivamonte.yml rigels51 authored 5 days ago	Verified 520bddb
Merge pull request #3 from rigels51/rigel rigels51 authored 5 days ago	Verified 7177277
Merge branch 'rigel' of https://github.com/rigels51/ASD-Project-Group18 into rigel rigels51 committed 5 days ago · 1 / 3	061f266

Jeriko Arceo



Lazizbek Ismoilov

Commits cover: initial folder structure, database schema and seeding, Flask backend with CRUD and clash detection, AI-mode agent integration, frontend build and redesign to match the shared team theme, Dockerfile and docker-compose configuration, CI/CD workflow setup and fixes, the tab-panel visibility bugfix, and merge conflict resolution when integrating into the shared root docker-compose.yml.

main	Lazismoilov	All time
Commits on Sep 8, 2026		
Merge branch 'main' of https://github.com/rigels51/ASD-Project-Group18 Lazismoilov committed 45 minutes ago · ✓ 1 / 1	84834f2	<>
Fix missing tab-panel display rule — resolves role/tab visibility bugs Lazismoilov committed 46 minutes ago	bbcba2f	<>
Commits on Sep 4, 2026		
Re-skin frontend to match team's shared visual theme (ink/paper/brass tokens, drawer pattern for Create, pill components) Lazismoilov committed 4 days ago · ✓ 1 / 1	47853fb	<>
Merge pull request #12 from rigels51/student-3/timetable-integration  Lazismoilov authored 4 days ago · ✓ 1 / 1	Verified e32d53a	<>
Merge main into student-3/timetable-integration, resolve docker-compose.yml conflict Lazismoilov committed 4 days ago · ✗ 1 / 3	c1e4cf8	<>
Commits on Sep 3, 2026		
Document Timetable & Class Scheduling service in README Lazismoilov committed 5 days ago	e943609	<>
Expand student-3 README with full feature documentation Lazismoilov committed 5 days ago · ✓ 1 / 1	c0092d1	<>
Fix workflow path filter so editing the workflow itself triggers a run Lazismoilov committed 5 days ago · ✓ 1 / 1	39fcbe0	<>
Add CI/CD workflow for Timetable & Class Scheduling microservice Lazismoilov committed 5 days ago	e76aafa	<>
Add Timetable & Class Scheduling microservice (Release 0)  Lazismoilov committed last week	3939216	<>

Yi Zhang

The GitHub commit history below provides evidence of my individual development work on Student 4, including frontend updates, workflow changes, feature integration and other

implementation changes.

rigels51 / ASD-Project-Group18

Type to search

<> Code Issues Pull requests Agents Actions Projects Security and quality Insights Settings

Commits

Yi-Zhang

All usersAll time

Commits on Sep 4, 2026

update

YiZhang2004 committed 4 days ago ✓ 1 / 1

2ad47c9

<>

Commits on Sep 3, 2026

update ui

YiZhang2004 committed 4 days ago ✗ 3 / 4

78de6f4

<>

Update index.html

YiZhang2004 committed 4 days ago ✗ 3 / 4

aa8592f

<>

Update YiZhang.yml

YiZhang2004 committed 4 days ago ✓ 3 / 3

4eb770f

<>

Merge branch 'Yi-Zhang' of https://github.com/rigels51/ASD-Project-Group18 into Yi-Zhang

YiZhang2004 committed 4 days ago

e0a0d62

<>

1

YiZhang2004 committed 4 days ago

2244a9d

<>

Merge branch 'main' into Yi-Zhang

YiZhang2004 authored 4 days ago ✓ 1 / 1

Verifiede0973ee

<>

Update YiZhang.yml

YiZhang2004 committed 4 days ago

8a81ba2

<>

Update YiZhang.yml

YiZhang2004 committed 4 days ago

e4f5d2d

<>

Rename YiZhang.yml to YiZhang.yml

YiZhang2004 authored 4 days ago

Verified94633da

<>

Update YiZhang.yml

85dc299

r

<>

Vu Tien Thanh Nguyen

rigels51 / ASD-Project-Group18

Type to search

<> Code Issues Pull requests Actions Projects Security and quality Insights Settings

Commits

main

All usersAll time

Commits on Sep 8, 2026

student 5

ntienthanh authored and ntienthanh committed 28 minutes ago

70b1f2b

<>

Commits on Sep 3, 2026

Add my changes

ntienthanh authored and ntienthanh committed last week

d3ae711

<>

Commits on Aug 21, 2026

lab 4

jer0206 committed 3 weeks ago

7892bc6

<>

Initial commit

rigels51 authored 3 weeks ago

Verified82dd42f

<>

16.0 Contributions Logs (i)

Jeriko Arceo

My main contribution to the project was the Staff management Feature.

Contributions for the Application:

- I created the staff database and connected it to the backend API
- I designed the page's HTML and CSS
- I implemented an AI mode feature into the feature
- I designed the HTML for the main homepage
- I connected the Main homepage to each of the group's features

Contributions for the Report:

- Created the structure for the document
- Completed my individual feature allocation
- Completed the functional and non-functional requirements for my feature
- Completed the feature plan for my feature
- Completed the risk management plan for my feature
- Created the diagrams for the Overall project plan (Timeline, Gantt Chart, and milestones)
- Completed the individual software architecture diagrams for my feature
- Created the Integrated software architecture diagrams for section 5
- Completed the DevOps Pipeline Architecture diagram as well as the pipeline stage descriptions
- Completed Section 8. Agentic AI workflow
- Completed the GitHub Actions Workflow for my feature
- Created the showcase video for release 0

Lazizbek Ismoilov

My main contribution was the Timetable & Class Scheduling feature, built and integrated as an independent microservice from the ground up.

- Designed and implemented the SQLite schema and seed data for timetable sessions
- Built the Flask backend: full CRUD, automatic clash detection, and an AI-mode agent grounded in live timetable data via Ollama

- Built and redesigned the frontend twice — first a standalone design, then re-skinned to match the team’s shared ink/paper/brass CSS theme, including a drawer-based create form and pill-style status indicators
- Containerised both services with Docker, wrote a standalone docker-compose.yml for isolated testing, and integrated into the team’s shared root docker-compose.yml
- Set up and debugged the GitHub Actions CI/CD workflow (student-3.yml)
- Found and fixed several real bugs: Flask host binding for containers, CORS for cross-origin frontend/backend calls, htmx’s cross-origin AJAX restriction, a missing tab-panel CSS rule causing role/tab visibility issues, and Docker container name conflicts
- Resolved a live merge conflict in the shared docker-compose.yml when integrating alongside another student’s feature
- Validated the feature with a 16-check endpoint test suite and documented results in an evidence log

Yi Zhang

My main contribution was the Course and Enrolment Management feature.

I developed the course and enrolment database and implemented the main CRUD functions.

I created the front-end pages for Course Management and Student Enrolment.

I also added AI Mode using Ollama. The AI can answer general questions and use current course and enrolment data from the database.

I separated the feature into frontend, backend, and database services.

I also created the Docker configuration and GitHub Actions workflow for Student 4.

Rigel Rivamonte

My main contribution was the Student Records Management feature.

I designed the SQLite student database and added sample records to have for first time start-up. I implemented the Flask database API for student CRUD operations, search, lookup by ID, and course filtering within the page.

I developed the frontend student records interface with search, status filters, student details, forms, and AI chat. I also added the Flask application service that connects the frontend to the

database service. At the same time, I also integrated the Ollama AI Mode using current student records as context.

Vu Tien Thanh Nguyen

My main contribution was the **Assessment & Grades Management** feature.

Contributions for the Application:

- I designed and implemented the Assessment and Grade data structure.
- I developed the Flask database service using SQLite for storing assessment and grade information.
- I implemented CRUD operations for assessments and grades.
- I developed the Assessment & Grades frontend interface.
- I integrated the feature with Student 1's Student Records service to validate student IDs and retrieve student information.
- I integrated the feature with Student 4's Course Management service to retrieve course information.
- I implemented AI Mode using Ollama and the qwen2.5:0.5b model.
- I implemented the Plan → Act → Observe → Adapt workflow so AI questions can use current assessment and grade information.
- I containerised the frontend, backend, and database services using Docker.
- I integrated the Student 5 services into the group's shared Docker Compose application.
- I created automated tests for the Assessment & Grades API.
- I created the GitHub Actions CI workflow for Student 5.

Contributions for the Report:

- Completed my individual feature allocation.
- Completed the functional and non-functional requirements for my feature.
- Completed the feature plan for Assessment & Grades Management.
- Completed the risk management plan for my feature.
- Completed the individual software architecture description and diagram for Student 5.
- Documented the Student 5 GitHub Actions workflow.
- Added Student 5 implementation evidence, GitHub commit evidence, known issues, and limitations.

17.0 Showcase Video

YouTube link - youtube.com/watch?v=GSsBDXj-1N8&feature=youtu.be